

**QEC Compilation for Color Codes in
Quantum Charge-Coupled Devices**

Computer Science Tripos – Part II

2026

Declaration

I, the candidate for Part II of the Computer Science Tripos with Blind Grading Number 3575Z, hereby declare that this report and the work described in it are my own work, unaided except as may be specified below, and that the report does not contain material that has already been used to any substantial extent for a comparable purpose. In preparation of this report, I adhered to the Department of Computer Science and Technology AI Policy. The project required the approved use of an open source QCCD compiler and such use is acknowledged in the text at the relevant sections. I am content for my report to be made available to the students and staff of the University. Date September 6, 2026

Proforma

Candidate number: 3575Z

Project title: QEC Compilation for Color Codes in Quantum Charge-Coupled Devices

Examination: Computer Science Tripos – Part II, May 2026

Word count: 11721¹

Code line count: 4329²

Project originator: Dr Prakash Murali

Day-to-day and Marking supervisor: Dr Prakash Murali

Original aims of the project

The main objective of this project is to produce a color code optimised compilation pass onto quantum charge-coupled device (QCCD) architecture. The end goal is to address the limitations of existing QCCD compilers by expanding a restrictive compiler into a generalised framework capable of evaluating a larger range of QEC codes. The key implementation components are to build a suite of color code circuits, designing custom mapping heuristics that exploit the native geometry of color codes ideally outperforming existing heuristics, and adapting the compilation pipeline to improve the performance of this code class on QCCD devices.

Work completed

All project deliverables and intended extensions were completed. The project successfully produced a generalised color-code-compatible compiler that rivals the state-of-the-art Moveless compiler. Key technical contributions include the development of (previously unsupported) chromobius-compatible stim circuits and demonstrating that the novel, topology-aware mapping scheme outperforms generic heuristics on given hardware configurations. Moreover, an alternative (4.8.8) code is implemented, alongside an extension identifying optimal architectural parameters for the color code and a comparison of its performance against surface codes. To support further QEC-QCCD based research, the complete compiler and circuit repository is made open sourced.

¹Computed with Overleaf's TeXcount utility

²Computed via cloc (<https://github.com/AlDanial/cloc>)

Special difficulties

None.

Contents

1	Introduction	1
1.1	Context and Motivation	1
1.2	Related Works	2
2	Preparation	3
2.1	Quantum Computing Preliminaries	3
2.2	Quantum Error Correction (QEC)	4
2.2.1	From Classical to Quantum Error Correction	4
2.2.2	QEC Metrics and Properties	5
2.2.3	Stabilizer Codes	5
2.2.4	Surface Codes	5
2.2.5	Color Codes	6
2.2.6	Decoding	7
2.3	Trapped Ion Quantum Charge-Coupled Devices	7
2.3.1	Trapped Ions	7
2.3.2	Quantum Charge-Coupled Device (QCCD)	8
2.4	The Baseline Framework	9
2.5	Starting Point	9
2.6	Requirement Analysis	9
2.7	Software Engineering Techniques	10
2.7.1	Development Model	10
2.7.2	Libraries, licences and tools	10
2.7.3	Hardware and Backups	11
3	Implementation	12
3.1	Compilation Framework	12
3.2	Repository Overview	14
3.3	Implementing Color Code Circuits	15
3.3.1	(6.6.6) Color Code Topology	15
3.3.2	Generating the Circuit	16
3.3.3	Noise Model	16
3.3.4	Logical Observable	17
3.4	Qubit Mapping	18
3.4.1	Triangular Mapping Strategy	18

3.4.2	Preventing Overclustering	19
3.4.3	Edge Case Mapping Methodology	21
3.5	Testing and Debugging	21
3.6	Extension: The (4.8.8) Color Code	21
3.7	Extension: Research-based Experiments	23
3.8	Extension: Generalising Ion Routing	23
4	Evaluation	24
4.1	Evaluation Methodology	24
4.1.1	Architectural Setup	24
4.1.2	Clustering Heuristics	25
4.1.3	QEC codes	25
4.1.4	Metrics	25
4.2	Color Code Evaluation	26
4.3	Qubit Mapping	28
4.3.1	Clustering Effect on Compilation	28
4.4	Compiler Benchmarking	32
4.5	Analysis of Direct-Mapping	34
4.6	Architectural Space Exploration	34
4.6.1	Effect of Topology	34
4.6.2	Effect of Trap Capacity	36
4.7	Surface vs Color Codes	37
5	Conclusion	39
5.1	Review of Deliverables and Contributions	39
5.2	Reflections	39
5.3	Future Work	40
	Appendices	47
A	Supplementary Content	48
A.1	Alternative Ion Routing	48
A.1.1	Compilation Tests	49
A.2	Clustering Results	50
B	Project Proposal	52
B.1	Description of Project	52
B.1.1	Main Goals	52
B.1.2	Extensions	53
B.2	Evaluation	53
B.3	Success Criteria	53
B.4	Risk Management	53
B.5	Resource Declaration	53

B.6 Starting Point 54

B.7 Timetable 54

Chapter 1

Introduction

1.1 Context and Motivation

Quantum computing offers a fundamentally new paradigm for computation, promising super-polynomial speed-ups for specific classes of complex problems such as prime factorisation and quantum chemistry simulation [1, 2]. However, for bridging the gap between the error rates of near-term hardware (typically 10^{-3} to 10^{-4} [3, 4]) and the requirements for practical quantum applications (often below 10^{-10} [5]), robust Quantum Error Correction (QEC) is necessary. Popular QEC codes include the surface code, quantum repetition code and the Steane code. This work specifically focuses on topological QEC codes, where the code structure is defined on a lattice.

Trapped-ion (TI) qubits are a potential candidate for realising QEC, attributed to their high gate fidelities and long coherence times [3, 6]. Despite these advantages, TI systems face severe physical constraints. Most notably, they suffer from slow gate operation speeds; state-of-the-art two-qubit TI gates operate at approximately $1.6\mu\text{s}$, which is slower than neutral atoms ($1\mu\text{s}$) or superconducting platforms (e.g. 60ns on Rigetti’s Cepheus-1-108Q) [7, 8]. Furthermore, as qubit counts scale, TI systems face a severe control-wiring bottleneck, where the required control input lines per chip outscale cutting-edge traditional CPU packaging [9].

To overcome these scaling limitations, the Quantum Charge-Coupled Device (QCCD) architecture has emerged as the leading proposal for scaling TI systems. The architecture proposes a modular approach where ions can be dynamically “shuttled” (moved) through distributed ion traps, with leading industry examples being the 98-qubit Quantinuum Helios [10] and IonQ’s 100-qubit Tempo architecture [11]. Advancements in integrated wiring techniques even outline realistic pathways to scale beyond 1000 qubits [9]. Implementing practical-scale QEC codes on QCCD hardware introduces complex operational challenges. The inter-trap shuttling necessary to execute these parity checks can lead to significant latency and dephasing errors [12, 13]. As a result, achieving massive operational parallelism during circuit execution is critical to suppress logical error accumulation. Addressing these bottlenecks requires highly specialised, QEC-aware compilation frameworks.

Prior work on QCCD-based compilers often optimise under noisy intermediate-scale quantum (NISQ) assumptions of limited qubit counts and high operational noise; such compilers are designed for flexibility and are inefficient for QEC codes that have exploitable topological and structural traits [14–16]. Recent works explore QEC-aware compilation strategies onto QCCD hardware, yet many

solely optimise for the surface code [17, 18]. When applied to structurally diverse codes, they are incapable of compiling them or do so sub-optimally as they fail to exploit properties of alternative QEC codes. Notably, Moveless, a highly optimised QCCD compiler, supports the compilation of both surface and color codes; however its optimisations occur at the scheduling and routing level rather than utilising topological features of the codes [14].

This surface-code bias restricts the exploration of color codes¹, a highly promising alternative. Certain implementations rival the threshold of surface codes [20] and natively support a broader set of transversal logical gates, drastically reducing the software overhead required for fault-tolerant execution. Yet, mapping their complex, trivalent lattices and higher-weight stabilizers (e.g. weight-six or weight-eight stabilizers) onto hardware introduce massive shuttling penalties. Consequently, the efficient physical compilation of color codes onto QCCD hardware remains an open, highly challenging, and largely under-researched domain.

This project directly addresses this gap by adapting a surface-code-centric toolflow into a generalised framework capable of evaluating structurally diverse color codes on QCCD hardware. To overcome the inefficiencies of existing mapping heuristics, this work proposes and implements a novel, topology-aware clustering scheme catered specifically to the color code. By exploiting the natural geometry of the code during hardware mapping, the framework minimises shuttling bottlenecks and maximises parallelism. Ultimately, this toolflow enables benchmarking of color codes against existing baseline schemes, providing key architectural insights that can guide future software-hardware designs.

The resulting compilation pipeline reveals that switch-based architectures with small traps of up to two ions are effective, exhibiting code-distance agnostic constant cycle times and superior error suppression capabilities over alternative QCCD device topologies. Moreover, the proposed mapping scheme demonstrates an improvement of up to 3.47x in logical error rate over the baseline mapping in [17], pointing to the efficacy of a topology-aware color code compilation strategy that was previously unexplored by other compilers.

1.2 Related Works

Moveless [14] presents a highly optimised QEC compilation onto QCCD via instruction reordering, routing heuristics and ancilla reuse, alongside Murali et al. [15] examining the influence of QCCD topologies on performance, demonstrating the importance of software-hardware co-design for compilers and QCCD devices. Seminal work by Landahl et al. [21] on color codes, alongside the contributions of Gidney and Jones [22] and Takada and Fujii [23] on new color code circuits and custom decoders advocate for further research into this unique code family. Work by Ovide et al. [24] achieves significant performance gains in optimising initial qubit placement on QCCD traps, inspiring this project’s focus on tailoring the qubit placement for color codes to best facilitate error correction.

¹Using the US spelling as introduced in [19] and standard in literature

Chapter 2

Preparation

2.1 Quantum Computing Preliminaries

First, we begin with a background on quantum computing, quantum error correction and trapped ion hardware, to introduce and contextualise the concepts used throughout this project.

The Basics of Quantum Computing

Classical hardware encodes information in bits, as binary values of 0 and 1. In the case of quantum computers, information can be stored using quantum bits, “qubits”, allowing a qubit to be in a state of $|0\rangle$, $|1\rangle$ or in some linear combination (a superposition) $\alpha|0\rangle + \beta|1\rangle$ by leveraging quantum superposition. The values α and β are complex numbers referred to as probability amplitudes, where $|\alpha|^2$ and $|\beta|^2$ give the probability of finding the qubit in the $|0\rangle$ and $|1\rangle$ states respectively, additionally satisfying the normalisation condition of $|\alpha|^2 + |\beta|^2 = 1$.

The following key concepts govern the state and evolution of the states in a quantum computer.

1. The state of a quantum system is fully encapsulated by its state vector (a unit vector in the Hilbert space of the system)
2. Evolution of a state of a quantum state in discrete time can be described by a sequence of unitary (i.e $U^{-1} = U^\dagger$), norm-preserving operations
3. Measurement collapses a quantum state into the observed state

The state space of a qubit can then be represented as a unit vector. The second postulate ensures that the state of a qubit always satisfies the normalisation condition, and allows us to evolve the state of a qubit using unitary quantum gates. The Pauli matrices are such examples:

$$X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}, Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} \quad (2.1)$$

The X gate acts as a NOT operation, Y has the effect of $Y|0\rangle = i|1\rangle, Y|1\rangle = -i|0\rangle$ and Z reverses the parity (positive/negative) for $|1\rangle$ states. The Hadamard gate is frequently used to produce a

superposition state:

$$H = \frac{1}{\sqrt{2}} \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad H|0\rangle = |+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle), \quad H|1\rangle = |-\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle) \quad (2.2)$$

We define the universal gate set, a set of unitary gates that can be used to approximate any other unitary transformation to an arbitrary precision. A possible set includes the Hadamard gate H , the $CNOT$ (controlled-NOT) gate and the T gate.

$$CNOT = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix} \quad T = \begin{pmatrix} 1 & 0 \\ 0 & e^{i\pi/4} \end{pmatrix} \quad (2.3)$$

Clifford gates are defined as the elements of the Clifford group, mapping tensor products of Pauli matrices to tensor products of Pauli matrices with conjugation, where a, b in some group are conjugate if for some group element f , $faf^{-1} = b$. Notably, the Hadamard gate is a member as $HXH^\dagger = Z$ and $HZH^\dagger = X$, as is the $CNOT$ (e.g. $CNOT(X \otimes I)CNOT^\dagger = X \otimes X$).

2.2 Quantum Error Correction (QEC)

2.2.1 From Classical to Quantum Error Correction

Classical error correction relies on redundancy. For instance, a 3-bit repetition code encodes a ‘0’ as ‘000’. If a bit flip converts the message to ‘100’, the receiver utilises a majority vote to recover the original ‘0’.

Quantum error correction (QEC) requires an alternative approach, since measuring a quantum state collapses its superposition, preventing us from examining the qubits directly. Instead, the quantum 3-bit repetition code (e.g., $\alpha|0\rangle + \beta|1\rangle \rightarrow \alpha|000\rangle + \beta|111\rangle$) introduces auxiliary “ancilla” qubits to act as an intermediary.

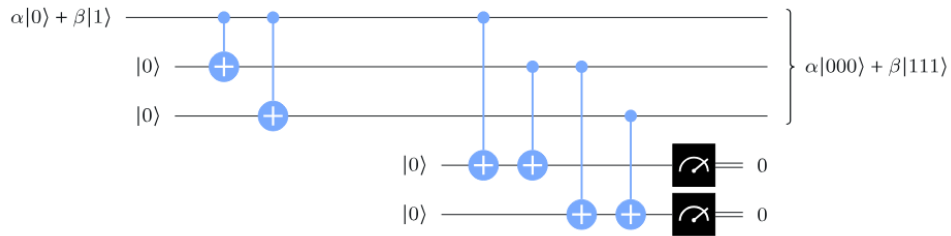


Figure 2.1: A 3-qubit repetition code circuit from Ref. [25] demonstrating the use of two ancilla qubits to detect parity flips in the encoded state

As shown in Figure 2.1, the ancillas interact with the data qubits via $CNOT$ gates to check their parity. Measuring the ancillas produce “error syndromes”—outcomes that pinpoint if and where a bit flip occurred, leaving the encoded data intact. This ancilla-based syndrome extraction circuit can

be adapted to instead detect phase-flip errors [25]. The concatenation of these ancilla-based error correction circuits acts as a pathway to scaling QEC.

2.2.2 QEC Metrics and Properties

When evaluating QEC codes, the primary performance metric is the fault-tolerant threshold. If the hardware’s physical error rate is below this threshold, increasing the size (or “distance”) of the code exponentially suppresses the logical error rate [26, 27]. Conversely, if the hardware noise exceeds the threshold, making the code larger only introduces more errors. Consequently, a code with a higher threshold is always more desirable.

Another ideal property of QEC codes are transversal gate implementations. Transversal logical gates are implemented by applying a gate to each individual physical qubit within a logical qubit, containing the propagation of gate errors within each physical qubit. Therefore, transversal gate implementations are significant in suppressing the logical error rate. The Eastin-Knill Theorem states that no QEC code can implement a (unitary) universal gate set transversally [28]. Practically, this means that codes with a natively large set of transversal gate operations are ideal, using methods such as lattice surgery or magic state distillation to enable further transversality [29–31].

2.2.3 Stabilizer Codes

Stabilizer codes¹ are a class of quantum codes used for error correction, defined through a set of stabilizer generators that dictate a valid set of states — its “codespace”.

A stabilizer generator is defined to be an n -qubit Pauli operator $P \in \Pi^n$ such that all valid states/codes $|\psi\rangle$ in the codespace have a +1 eigenvalue: $P|\psi\rangle = |\psi\rangle$. Stabilizer codes can be realised into quantum circuits by measuring the corresponding Pauli operators, using ancilla qubits to hold the resulting eigenvalue or “error syndrome” [32].

Calderbank-Shor-Steane (CSS) codes are a family of stabilizer codes where stabilizer generators can be expressed solely with the Pauli X and Z operators, with the condition that X and Z never appear in the same stabilizer generator [33]. A CSS code is topological if it can be embedded on a surface.

2.2.4 Surface Codes

The surface code represents a class of 2-D topological CSS codes. A single logical qubit is encoded across a two-dimensional planar lattice of physical data qubits. Error detection is performed through a chequerboard pattern of alternating X-type and Z-type stabilizers, with each stabilizer measuring the parity of adjacent data qubits.

The code distance, d , is defined by the number of physical qubits along the shortest boundary of the lattice. This distance determines the code’s error-correcting capability, as a higher distance enables higher logical error suppression under physical noise below the surface code’s threshold.

While traditionally defined on a standard orthogonal grid, modern literature predominantly utilises the rotated surface code (Fig. 2.2). This variant, defined on a grid rotated by 45° , significantly

¹Using the US spelling throughout this dissertation

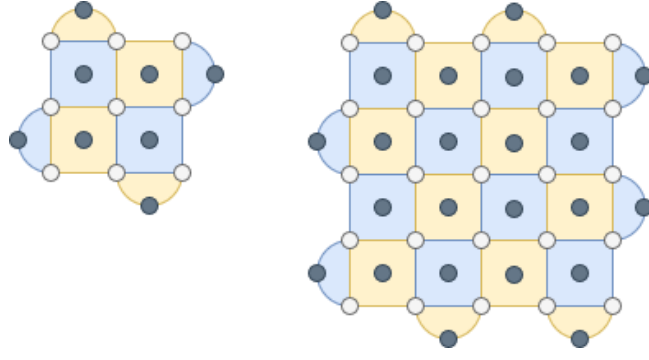


Figure 2.2: 2D rotated surface codes at distances 3 and 5. Data qubits (white) are defined on the vertices of each surface, with ancilla qubits (grey) in the centres. The surfaces are shaded yellow or blue to differentiate the X and Z stabilizers respectively.

reduces the physical qubit overhead required to achieve a target logical error rate, making it the highly favoured baseline for hardware implementations [34].

Surface codes currently dominate QEC research, attributed to their simple topology and high thresholds of around 0.5-1.1% [35, 36]. However, to realise large-scale fault-tolerant quantum computing (FTQC), the universal gate set must be implemented in a fault-tolerant way, which the surface code currently struggles with. Specifically, the surface code lacks a native, transversal implementation for non-Clifford operations such as the T gate, forcing surface codes to rely on alternative methods such as magic state distillation or lattice surgery. Such methods often introduce a large space-time implementation overhead and can come at the cost of reducing the surface code's error suppression capabilities [37, 38]. Additionally, space-time costs for Clifford gates with magic state distillation can be up to $O(d^3)$ with distance d [31].

2.2.5 Color Codes

Color codes constitute a class of 2D topological CSS codes defined on a three-colourable, three-valent, uniformly tiled lattice. Each face f in the lattice defines two stabilizer generators in terms of Pauli operators acting on the vertices of that face: $S_X^f = \bigotimes_{v \in f} X_v$, $S_Z^f = \bigotimes_{v \in f} Z_v$.

The color code preserves the state of the logical qubit by repeated stabilizer measurements, with the outcomes (syndromes) compared between rounds, such that a change in parity indicates an error.

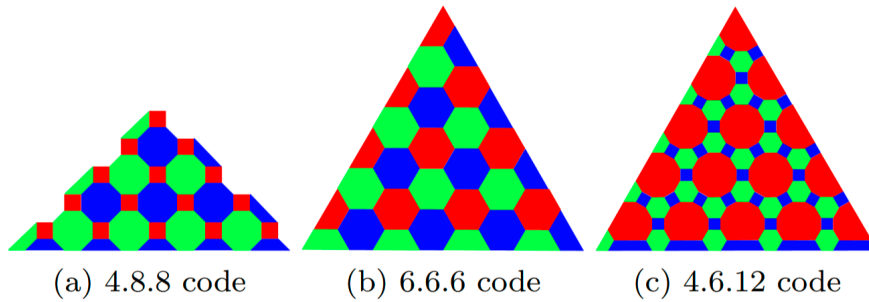


Figure 2.3: The three possible tilings of a triangular, distance 11 color code from [21].

This project specifically examines the compilation of triangular color codes [19] (examples in Fig. 2.3), which encode a single logical qubit. The spatial locality of the quantum operations, where

interactions are restricted to neighbouring tile qubits, suggests that these codes may benefit from a topology-aware mapping scheme, following similar intuitions of past work on surface codes [17]. This project asks: *How could the features of the color code be utilised effectively in compilation?*

Compared to surface codes, color codes often have higher-weight stabilizers, as each ancilla qubit interacts with a larger number of data qubits during measurement. As a result, circuit depth (sequential time steps to perform circuit operations) increases, introducing more errors and results in lower thresholds.

However, a key advantage of color codes is their ability to implement all logical Clifford gates transversally [30]. This property has led color codes to be considered as promising candidates for FTQC with lower implementation overheads than surface codes, which lack this feature.

2.2.6 Decoding

To protect the information of a logical qubit using a QEC code, a decoder is used to take the syndrome/parity measurement information as input, inferring the most likely physical errors that occurred during computation to provide a correction.

Stim, a QEC-specialised Python library, makes use of detectors that are defined between syndrome measurement rounds, used to flag errors when the parity information between adjacent detectors differ [39].

QEC code implementations in `stim` will often make use of decoders such as `pymatching`, which simplifies decoding to a minimum perfect weight matching problem on a matching graph. This graph is constructed with detectors as nodes and the edges between nodes representing potential errors that flip those detectors, weighted with the logarithm of the probability of the error [40].

Color codes cannot be directly decoded with `pymatching`, as one error often sets off three or more detection events rather than two, which prevents these “hyper-edge” errors from being mapped directly into the matching graph. Instead, this project will use `chromobius`, an open source decoder based on a Möbius decoder [41]. `chromobius` utilises additional information on the color code circuit provided through detector annotations. Errors are then mapped into Möbius errors, translating the problem into a Möbius matching graph [22].

A key part of this project relies on integrating support for `chromobius` into the QCCD architecture simulation, allowing QCCD-compiled color code circuits to be decoded. Without this critical step, experimentation and evaluation of the color code on hardware using the framework is impractical or even impossible at higher code distances.

2.3 Trapped Ion Quantum Charge-Coupled Devices

2.3.1 Trapped Ions

Trapped Ion (TI) quantum computers make use of charged atomic particles (ions) as qubits, suspending the ions in space using electromagnetic fields. Some key advantages of TI systems are identical qubits, all-to-all connectivity, high coherence times [6] and the potential to outperform other architectures such as superconducting systems of the same size [42].

2.3.2 Quantum Charge-Coupled Device (QCCD)

QCCD (Quantum Charge-Coupled Device) architecture was first proposed in 2002 [43] in response to the practical challenges of manipulating large numbers of ions within a single trap, which limited scalability [44, 45]. Rather than confining all ions into a single region, QCCD adopts a modular design in which ions are distributed across multiple interconnected traps. Computation is done through ion transport, where ions are “shuttled” between trapping zones to enable quantum gate operations and entanglement.

The primitive QCCD operations assumed in this project (from [17, 29]) include an entangling Mølmer-Sørensen (MS) gate (t1), single-qubit rotations (around the x,y and z axes) (t2-t4), measurement (t5) and qubit reset (t6). Movement primitives include the movement of ions across a transport segment (t7), split from/merge into a trap (t8,t9) and entry/exit from a junction (t10,t11). For an ion to split from a trap, it must be at the end, performed using a SWAP operation (three 2-qubit operations (t1)) on the ions within the trap.

While the original proposal suggests an explicit separation between memory and interaction regions in the architecture, more recent experimental implementations and compiler frameworks have relaxed these strict zone assumptions [16]. QCCD architectures have since been realised by several groups, including IonQ [11] and Quantinuum [46].

An example QCCD architecture is depicted in Fig. 2.4, illustrating the components that form the architectural topology. This project will utilise this abstraction as the basis for QCCD-related components in this report.

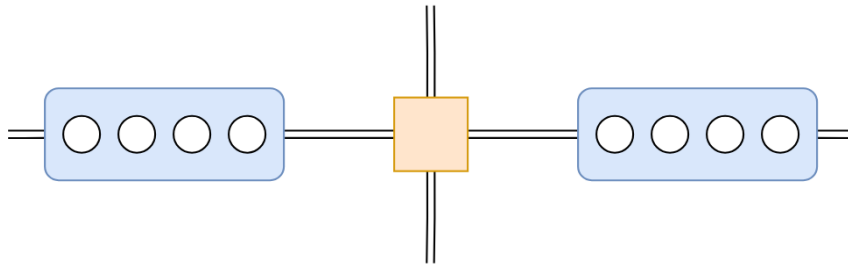


Figure 2.4: QCCD architecture showing 4 ions (white) in each trap (blue). The traps are connected via transport segments (black) and junctions (orange), where ions can be transported through for inter-trap movement.

Alternative QCCD topologies such as linear-chains and the race-track are seen in Quantinuum’s H1 and H2 series, respectively [46, 47], alongside experimental ring or hexagonal topologies proposed in QCCD-based compilers [14, 48]. The architecture of the device can heavily influence the performance of a QEC code, and the question of finding the best topology for scaling QEC systems remains an open challenge. This project aims to explore two key questions: *How do different architectures influence the performance of a color code?* *How can architectural parameters be optimised for the color code?*

2.4 The Baseline Framework

The existing compilation framework provides a highly optimised, state-of-the-art pipeline for mapping surface codes onto QCCD trapped-ion architectures [17].

However, its current software architecture is tightly coupled to the specific structural and topological properties of the surface code. For example, in the qubit-ion mapping, the heuristic performs an iterative bisection over a bounded square area, assuming the square-like structure of the code. The ion-routing algorithm strictly assumes ancilla-data interactions, such that alternative QEC schemes, such as the middle-out circuit with ancilla-ancilla operations, cannot be compiled [22]. These surface-code-centric assumptions highly restrict the compiler's ability to evaluate a wider range of QEC codes.

Therefore, the primary motivation of this project is to elevate the existing compiler from a hyper-specialised tool into a generalised, extensible framework for evaluating QEC codes on QCCD-based TI systems. To drive this architectural shift, this project specifically targets the compilation of quantum color codes.

This work intends to expand the capabilities of the current compiler by not only enabling the compilation of color codes onto QCCD hardware but also aims to establish a scalable pipeline capable of evaluating a broader class of quantum error-correcting codes, alongside exploring methodologies for further future generalisations.

2.5 Starting Point

Basic understanding of surface codes and decoders came from prior experience with quantum computing, primarily through a QEC summer workshop at Riverlane. Knowledge from Part IA Algorithms were useful when working on certain mapping algorithms, and the content in Part II Quantum Computing helped to cement my understanding of the theory. During the summer, I prepared for the project by undertaking the Google Quantum AI Quantum Error Correction course. I also spent some time familiarising myself with the existing compilation framework (Ref. [49]), experimenting with surface code compilation onto QCCD architecture. I began the project with a very elementary understanding of `stim` and other related python libraries, alongside limited knowledge of quantum computing platforms such as QCCD. I also started the project with no prior knowledge of group theory and stabilizer formalism used in defining QEC codes. No code was written before the beginning of Michaelmas term.

2.6 Requirement Analysis

The end goal, as stated in my project proposal (Appendix 5.3), is to build and demonstrate functionality of a compiler pass that successfully maps a color code circuit to QCCD hardware. To ensure the project remains on track, the main deliverables required to achieve the base goal and the extensions were clearly outlined before the implementation stage (Table 2.1).

Tasks of high complexity or those required by other deliverables were given high priority, whilst extensions were the lowest priority. The difficulty of a task is quantified by the amount of physical (coding) or theoretical (research/theory) work required. For example, identifying mapping mechanics

Main Deliverables	Priority	Difficulty
Identify candidate color codes	High	Medium
Identify potential mapping mechanics	High	Medium
Chromobius compatible color code implementation	High	High
Mapping algorithmic implementation	High	High
Integration of Chromobius into the compilation framework	Medium	Low
Building compilation pass for color codes	High	Medium
Testing and benchmarking compiler passes	Medium	Medium
Logical error rate evaluation	Medium	Low
Additional support for further tessellations (*)	Low	High
Investigation into ideal QCCD hardware parameters and topology (*)	Low	Medium
Comparison of surface and color code performance (*)	Low	Medium
Generalised ion routing (*)	Low	High

Table 2.1: Main deliverables of the project and their perceived risk/difficulty of implementation, with extensions denoted with (*)

requires a thorough understanding of current compiler methods and color code properties, placing its difficulty as medium.

2.7 Software Engineering Techniques

2.7.1 Development Model

The development model that will be employed in this project is the spiral model. Due to the high implementation difficulty, it is unlikely that the project can be completed in a single implementation pass. Instead, a risk-driven iterative process that repeatedly evaluates and refines the prototypes for the compilation stages is more suitable for this project. This approach ensures that I can carry out rigorous testing and validation with high-risk components of the project, an example being the color code circuit construction that requires careful analysis to ensure a valid circuit is achieved.

2.7.2 Libraries, licences and tools

The primary language used for development is Python 3.11, based on the main language of the compilation framework [17] and its widespread use in quantum computing tools [22], [39]. JupyterNotebook was also used to provide an interactive Python environment for experiments and testing. To streamline the testing and experimentation process, Python's `concurrent.futures` module is used to parallelise the execution over several compilation parameters (reducing total simulation times), and Windows batch files were written to automate workflows.

The framework used as a base for this project is open source. Table 2.2 also lists the key third party libraries and their corresponding licences used in the implementation with additional dependencies and libraries included in a `requirements.txt` file. All licences in the table are permissive licences, allowing free use, compilation, modification and distribution of the original source code. Project development was done inside a virtual environment (using python’s `venv`) to contain and isolate the Python packages required for the project.

Library	Purpose	Licence
<code>stim</code>	Simulation and analysis of QEC circuits	Apache 2.0
<code>sinter</code>	High speed Monte Carlo sampling of QEC circuits	Apache 2.0
<code>chromobius</code>	Color code circuit decoder	Apache 2.0
<code>pytest</code>	Framework for functional testing	MIT licence
<code>numpy</code>	Provides support for fast linear algebra computation	BSD licence
<code>matplotlib</code>	Enables visualisation of hardware and graph plotting	PSF licence
<code>networkx</code>	Provides standard graph algorithms used in routing and architecture representation	BSD licence
Jupyter Notebook	Interactive python notebooks for experiments	BSD licence

Table 2.2: Key third-party software and libraries used in the project

2.7.3 Hardware and Backups

Development took place on my personal laptop (ThinkPad T430, 2-core CPU, 8GB RAM, Windows 10 and Linux) and my custom PC (6-core AMD Ryzen 5 7600X, 32GB RAM, Windows 11) for more intensive tasks, both using VSCode IDE.

GitHub and git were used for version control and to make regular backups of local progress to an online repository. Documents and notes relating to the project were written and organised using Notion. Overleaf was used to write this dissertation and to keep a backup copy online.

Chapter 3

Implementation

This chapter breaks down the implementation details for the project. Beginning with the high-level architecture of the color code compilation pipeline (Section 3.1) and the repository overview (Section 3.2). The chapter proceeds to describe the in-depth theory and design-choices of the two critical deliverables: the color code implementation (Section 3.3) and the mapping process (Section 3.4). Section 3.5 details the main testing strategies, followed by the details of three subsequent extensions.

3.1 Compilation Framework

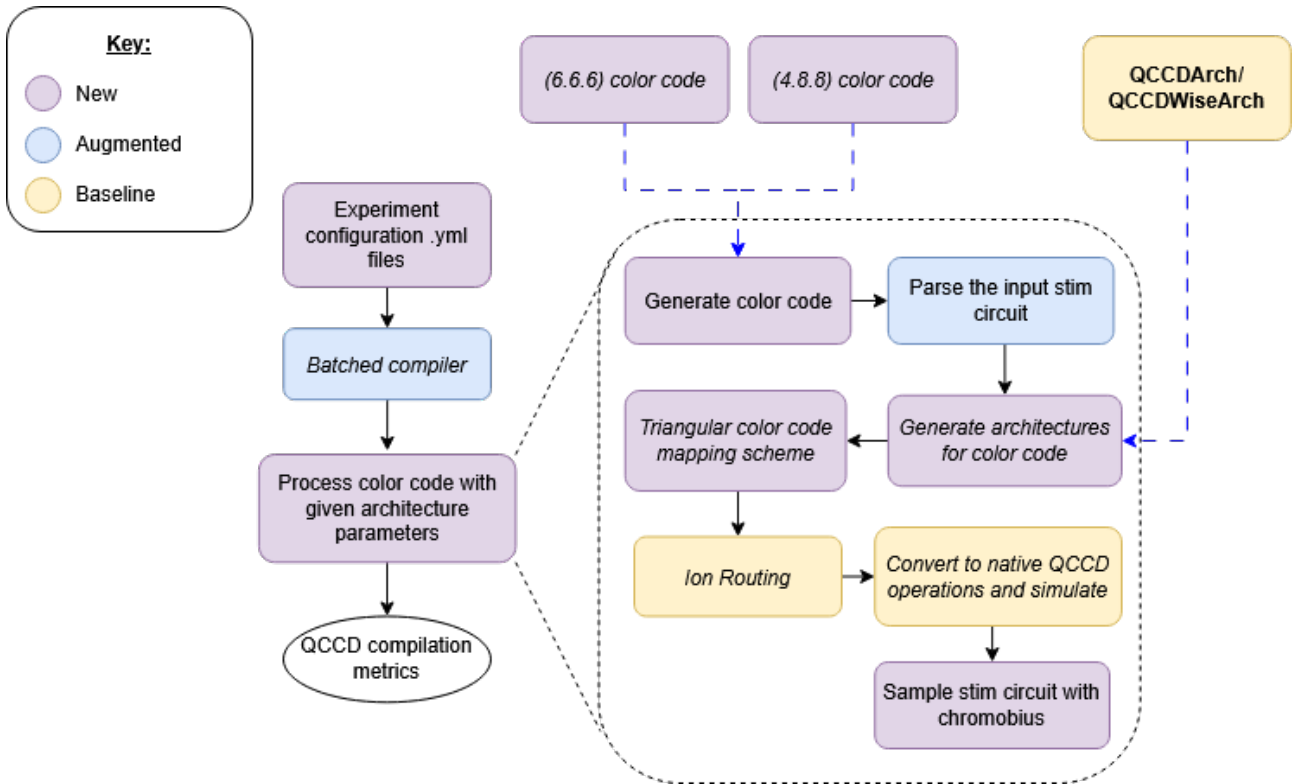


Figure 3.1: High-level architectural diagram of the control flow in the compilation pipeline of a color code, coloured according to the level of independent contribution. Black arrows indicate control flow, whilst blue arrows denote a “used in” relation.

Figure 3.1 details the stages in the compilation of a color code, highlighting critical sections that

were previously unsupported in the surface-code-centric compiler. Key stages include the use of the color code implementations from Section 3.3 in the generation stage, alongside the work in Section 3.4 for mapping the color code qubits to hardware ions.

Experiment variables are passed into `main.py` (batched compiler), which begins parallel/batched experiments over the given architecture and code parameters. Each compilation pass calls into the color code pipeline outlined by the dashed box, generating the required code and architecture to simulate the compiled circuit.

With extensibility and ease of experimental configurability in mind, the framework was rewritten in a way that enabled seamless switching between desired QEC codes via the configuration file. The compiler was further developed to dynamically invoke the relevant processes within the pipeline for a given code. For example, the decoder in the circuit sampling/simulation uses `pymatching` as the default for surface codes, but automatically switches to `chromobius` for color codes.

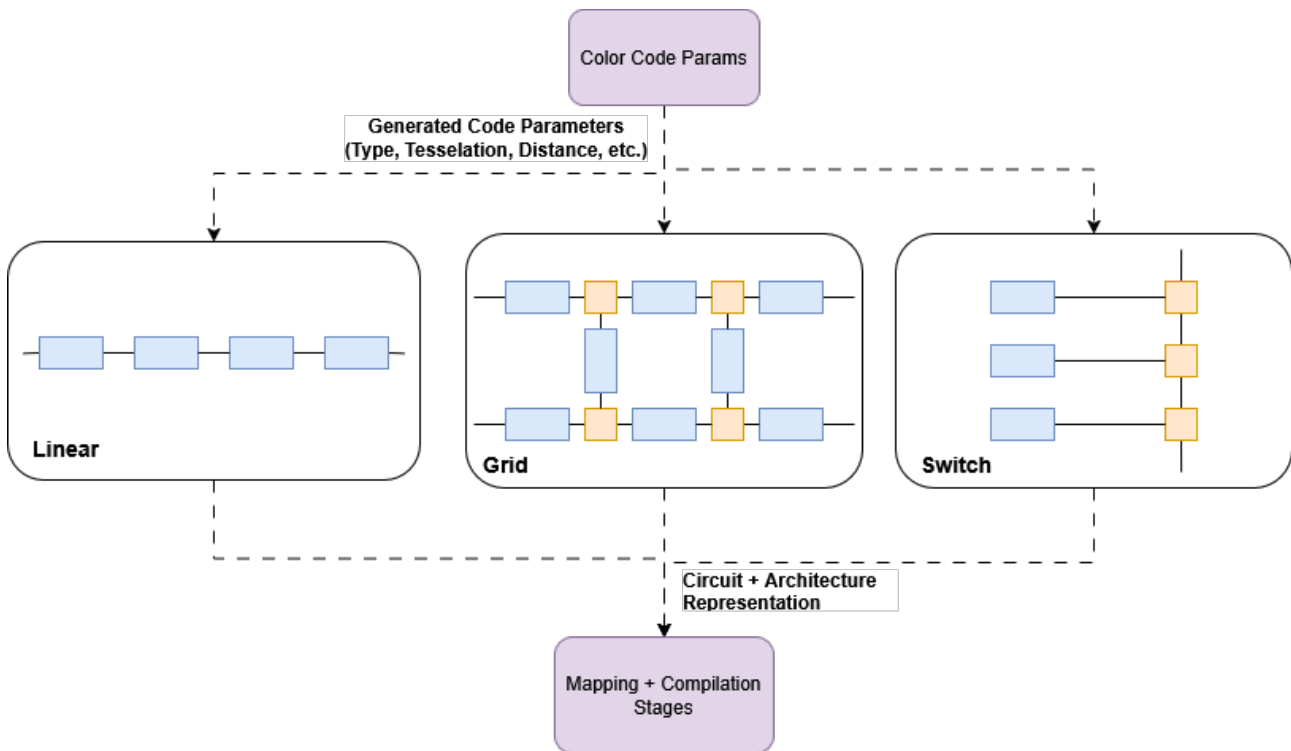


Figure 3.2: Depiction of the architectures built in the pipeline using the parameters of the code (which determine trap numbers, required width/height of grid, etc.).

Fig. 3.2 visualises the linear, grid and switch architectures that the color code circuits were compiled onto. Each architecture was built using the `QCCDArch` class by combining junction and trap objects, producing an architecture object that can be passed into the mapping stage alongside the color code circuit.

3.2 Repository Overview

```

CST-Part-II-Project/
├── src/
│   ├── compiler/
│   │   ├── qccd_color_qubits_to_ions.py .... topology-aware color code mapping
│   │   │   [427]
│   │   ├── qccd_alt_ion_routing.py ..... generalised ion routing algorithm [184]
│   │   └── simulator/
│   │       ├── qccd_circuit.py(*) ..... simulating compiled circuits [324]
│   │       ├── color_code_processor.py ..... process and compile color codes [289]
│   │       └── color_code_utils/
│   │           ├── color_code_circuits/ .. color code implementations (6.6.6, 4.8.8) [549]
│   │           ├── abstract_color_code_circuit.py
│   │           └── color_code_tile.py
│   ├── color_code_experiments/ ..... Jupyter notebooks for experiments and plotting
│   │   [1060]
│   │   └── plots/
│   ├── data/ ..... simulation results and outputs
│   ├── tests/ ..... functional/integration tests [451]
│   ├── configs/ ..... config .yaml files for simulation parameters
│   ├── logs/
│   ├── scripts/ ..... batch data collection scripts [273]
│   ├── main.py(*) ..... start of the compilation pipeline [98]
│   ├── README.md
│   ├── Makefile
│   └── requirements.txt

```

Figure 3.3: Structure of the repository, excluding content that is not strictly altered or developed entirely by myself. Pre-existing files that were augmented are denoted with (*). Line counts (counting only my contributions) for key files are enclosed within square brackets [-].

For the purposes of code submission, the work done in JupyterNotebook was condensed into a python file format. The line count on `color_code_experiments/` represents this number as opposed to the true line count of all notebooks. Submitted files are annotated to indicate and differentiate original work and pre-existing work via comments.

3.3 Implementing Color Code Circuits

The first milestone of this project was to produce a set of color code circuits that could be used to compile onto QCCD architecture. Motivated by the lack of native color code circuit support in `stim`, such as in the `qLDPC stim` circuit repository [50], the project required developing an alternative, new set of chromobius compatible color code circuits for the compilation pipeline.

3.3.1 (6.6.6) Color Code Topology

As mentioned in the preparation chapter, color codes require a three-colourable tiling that can manifest as 3 possible tessellations. The (4.8.8), the (6.6.6) and the (4.6.12), each defining a distinct triangular color code topology [21] (each tiling shown in Fig. 2.3).

Of these, the (6.6.6) was an ideal candidate for implementation due to its consistent topology and richer source of documentation and literature, enabling good progress towards a working prototype. Particularly, this tessellation also appears in the Moveless compiler’s test suite (although implemented in Quantum Assembly Language (`qasm`)), making it a suitable implementation choice for benchmarking in the evaluation stage [14].

The (6.6.6) triangular color code produces a uniform tiling made of hexagons and half-hexagons, bounded within an equilateral triangle. For this, a `ColorCodeCircuit666` class was developed, alongside a function `_generate_layout()` constructing the required hexagonal lattice for a given distance, defining data qubits at each vertex and a central ancilla qubit for each plaquette.

A specific design decision in this implementation was the use of a single ancillary qubit per face for stabilizer measurement (demonstrated in Fig. 3.4) as opposed to dual-ancilla approaches [51]. Although this approach doubles the syndrome extraction circuit depth, requiring one ancilla to measure both the X and Z stabilizers, it was chosen to provide ease of debugging and incremental validation during development. Notably, conceptually similar ancilla reuse methods are backed by QCCD compilers such as Moveless [14].

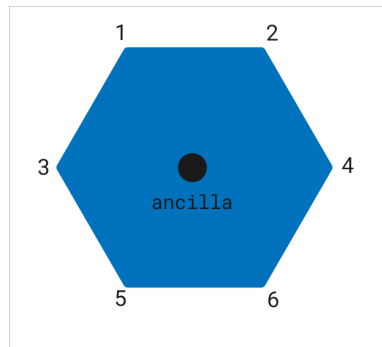


Figure 3.4: Example hexagonal plaquette and numbering corresponding to circuit schedule in Fig. 3.5

There exist more sophisticated scheduling strategies and architectural implementations achieving improved fault-tolerance thresholds [22, 23]. Such methods use novel ancillary or structural optimisations to protect against multi-qubit hook-errors that single-ancilla circuits are prone to. However, the objective for a working prototype naturally favoured a more foundational, robust implementation that could be developed from first principles.

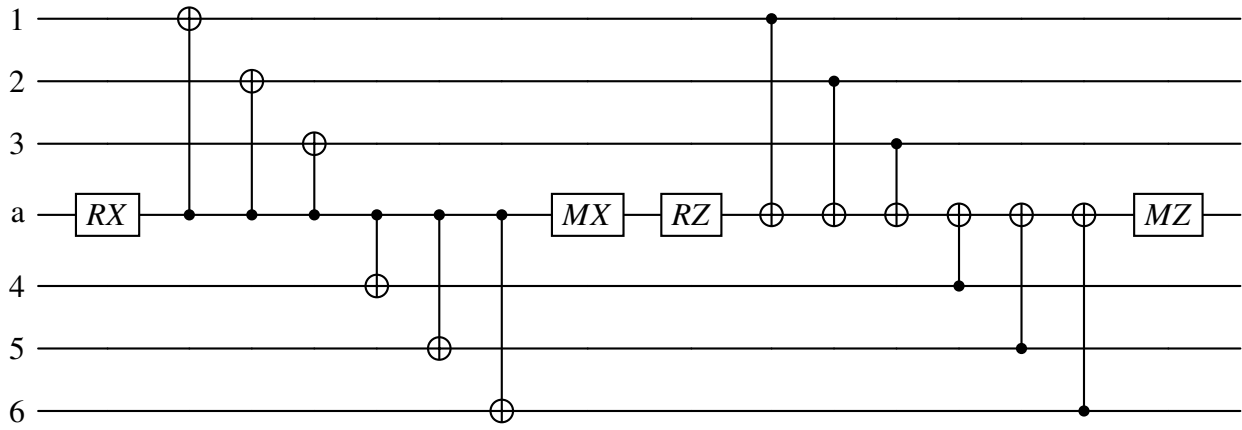


Figure 3.5: Measurement schedule for a color code circuit with a single ancilla qubit

3.3.2 Generating the Circuit

Typically, a stabilizer code circuit involves iterations of stabilizer measurement and defining detectors between cycles (demonstrated in Fig 3.6). Detectors in `stim` are used to provide explicit annotations that define which combination of measurement results should be parity checked to find the presence of an error. This enables measurement outcomes to be compared between consecutive rounds in the circuit such that detection events (changes in the stabilizer values) can be identified, suggesting the occurrence of errors.

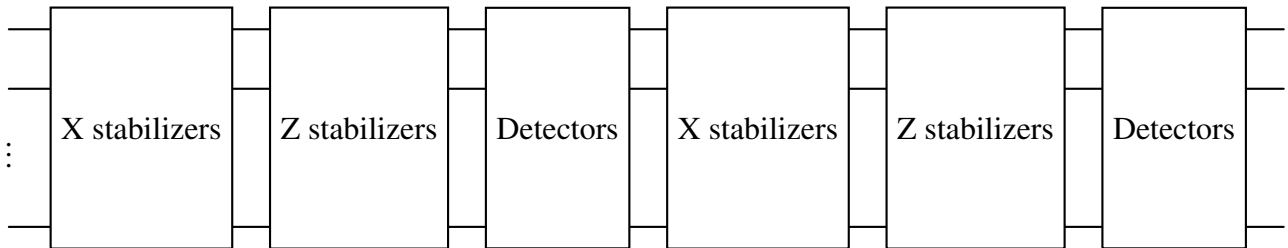


Figure 3.6: Example timeline of measuring the X and Z stabilizers in a color code.

Taking the tiles generated from the `_generate_layout()`, `_build_circuit()` was written to construct the `stim` circuit, using helper functions `_measure_rounds()` and `_measure_stabilizers()` to model the qubit-level operations per stabilizer measurement and include the detector annotations required by `chromobius` into the `stim` circuit. To guide my construction process, Refs. [21, 23] were examined for their single-qubit syndrome extraction models.

3.3.3 Noise Model

The noise model for a quantum circuit is used to characterise the types and distribution of errors that may occur during the operation of the circuit, mapping ideal unitary operations to noisy channels.

With a focus on realistic modelling, the noise model chosen for this implementation was the circuit-level noise model, selected to capture the detailed error mechanisms that may occur during the execution of a circuit. Unlike higher-level abstractions such as phenomenological or code-capacity noise,

this model assigns independent error probabilities to every physical operation, making it suitable for realistic hardware evaluation.

The circuit-level noise model of strength p is integrated directly into the circuit generation using the following `stim` operations:

- **Single-qubit operations (and idles):** Followed by a single-qubit depolarising channel $\varepsilon_1(p)$ via `DEPOLARIZE1(p)`. This applies a Pauli X , Y , or Z error, each with a probability of $p/3$.
- **Two-qubit gates:** Followed by a two-qubit depolarising channel $\varepsilon_2(p)$ via `DEPOLARIZE2(p)`. This applies any of the 15 non-trivial Pauli pairs ($P_1 \otimes P_2$) with an equal probability of $p/15$.
- **SPAM errors:** State preparations and measurements are subjected to bit-flip errors with probability p , implemented via `X_ERROR(p)`, `Z_ERROR(p)`, and `M(p)/MX(p)`.

The color code class was extended with this noise formalism, enabling a color code object to be optionally initialised with an argument to include circuit-level noise of strength p into the circuit. This framework enables the Monte Carlo `stim` simulations required to extract the logical error rates and validate the code construction in the Evaluation section.

3.3.4 Logical Observable

To empirically calculate logical error rates, QEC simulation frameworks such as `stim` require the explicit definition of a logical observable [39]. In the context of a simulation circuit, this observable is tracked via a specific set of physical qubits, allowing the simulator to verify whether the encoded logical state has been preserved or corrupted at the end of the execution. Theoretically, this set corresponds to the physical qubits that support a valid logical operator, geometrically represented as a contiguous string across the color code lattice.

Mathematically, in a stabilizer code defined by a stabilizer group $\mathcal{S}$, a logical observable is a Pauli operator $\bar{L}$ that satisfies two strict conditions. First, it must commute with all stabilizers in $\mathcal{S}$, meaning it belongs to the normaliser group $N(\mathcal{S})$. Second, it must be non-trivial; it cannot act as the logical identity I_L on the codespace. Consequently, a valid logical observable must reside strictly in the quotient group $N(\mathcal{S}) \setminus \mathcal{S}$ [52].

For topological codes, these logical operators can be deformed into strings of Pauli operators that span the boundaries of the code. As detailed in Ref. [29], the logical X_L operator, for example, in a triangular color code can be deterministically supported along any of its boundaries. Formally, this is defined as:

$$X_L = \bigotimes_{i \in \mathcal{V}(e)} X_i, \quad (3.1)$$

where $\mathcal{V}(e)$ denotes the set of vertices (physical qubits) along an arbitrary edge e of the triangle. Using this geometric property, the logical observable for the (6.6.6) implementation in this project is explicitly defined as the qubits comprising the bottom boundary of the lattice:

$$\mathcal{O}_L = \{q \mid q \in \mathcal{V}(e_0)\}, \quad (3.2)$$

where e_0 represents the bottom edge of the triangular code.

3.4 Qubit Mapping

Once the circuit is generated, the logical qubits must be mapped to physical QCCD traps via a two-step pipeline: qubit clustering, followed by physical trap assignment. Because the primary architectural challenge of the color code lies in its complex triangular topology and high-weight stabilizers, this implementation focuses exclusively on developing topology-aware clustering heuristics. The subsequent hardware-specific trap assignment is delegated to the base framework’s hill-climbing search [17], isolating the clustering challenge and avoiding “reinventing the wheel”.

Abstractly, clustering can be modelled as a balanced graph partitioning problem on $G = (V, E)$. The vertices V (qubits) must be divided into capacity-constrained sub-graphs while minimising the number of cut edges E (inter-trap routing operations). Because balanced partitioning is NP-complete [53], exact solvers such as SAT methods cannot scale to practical fault-tolerant code distances. Consequently, this framework employs a scalable, heuristic-based methodology that directly exploits the color code’s native geometry to preserve local qubit neighbourhoods.

Choosing an appropriate cluster capacity is critical at this stage, as this influences the ion routing schedule. Fully populating a trap eliminates the free space required for ion shuttling, whereas overly sparse traps inflate global routing overhead. To balance this trade-off, this project restricts the maximum cluster size to **trap capacity** – 1, mirroring the constraints established in Refs. [14, 17], whilst others relax this constraint to leave room for 2 incoming ions [15, 16]. This condition guarantees at least one open slot per trap, facilitating ion movement during parallel syndrome extraction.

3.4.1 Triangular Mapping Strategy

The underlying framework can support planar error correction codes such as surface codes, achieved through an efficient top-down, regular partitioning of a square bounded area. Whilst a sensible heuristic for the square-shaped surface code, the approach disregards the color code’s unique topology, and as a result, may produce sub-optimal mappings. To address this, I propose a novel mapping heuristic that directly exploits the natural geometric topology of the color code. Inspired by the Sierpinski triangle, the algorithm first bounds the entire array of ions within a single large outer triangle. The space is then recursively subdivided into four sub-triangles (visualised in Fig. 3.8(a)). The division process is repeated on each sub-triangle until the number of ions within it falls below the hardware’s capacity threshold.

For a point in the triangle with vertices A , B and C , the barycentric coordinates can be given as a linear combination of these vectors $p = uA + vB + wC$, with the constraint that $u + v + w = 1$, $u \geq 0, v \geq 0, w \geq 0$. The subdivision of points occupying a triangular area can be performed by conversion of each point into barycentric coordinates, then classifying the point into a sector of the triangle (Fig. 3.8(b)). The core logic for this spatial clustering approach, outlined in Alg. 1, uses an iterative, queue-based breadth-first search to perform partitioning steps.

By defining partition boundaries strictly through edge midpoints rather than rigid geometric properties (e.g. fixed angles or side lengths), the clustering methodology maintains compatibility across arbitrary triangular lattice layouts.

Assuming a relatively uniform distribution of N input qubits, each subdivision partition the points

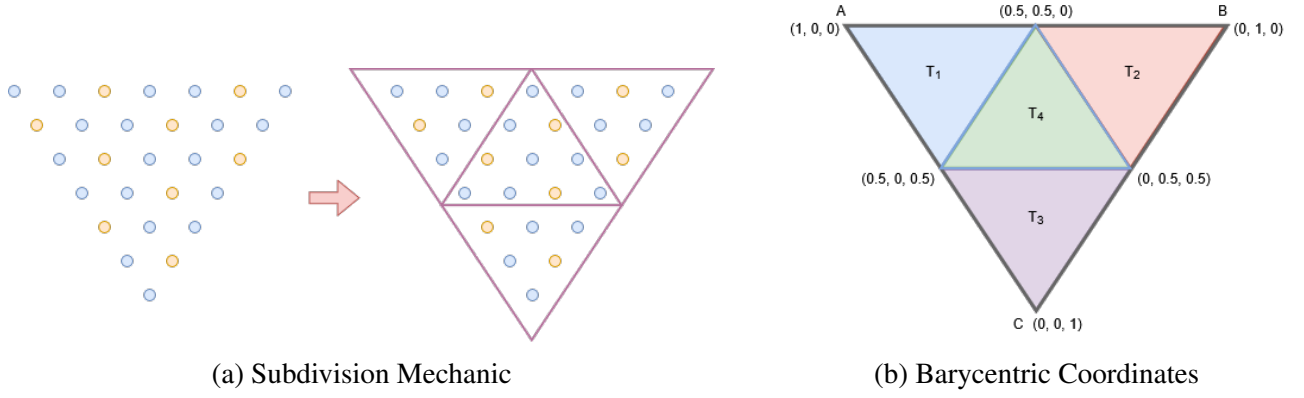


Figure 3.7: (a) Clustering mechanism for a triangular topology using an example distance 5 (6.6.6) color code with data qubits (blue) and ancilla qubits (yellow). (b) Sub-triangles defined within the barycentric coordinate space of the outer triangle, with points and their respective coordinates defined in terms of $\triangle ABC$.

Algorithm 1 Triangular Clustering Algorithm

```

1: Input: Point set  $X$ , capacity constraint  $k$ 
2: Output: Set of final clusters  $\mathcal{C}_{final}$  where  $\forall C \in \mathcal{C}_{final}. |C| \leq k$ 
3: Initialise active set  $\mathcal{A} \leftarrow \{(T_{root}, X)\}$  and clusters  $\mathcal{C} \leftarrow \emptyset$ 
4: while  $\mathcal{A}$  is not empty do
5:   Extract a tuple  $(T, P)$  from  $\mathcal{A}$ 
6:   if  $|P| \leq k$  then
7:      $\mathcal{C} \leftarrow \mathcal{C} \cup \{P\}$ 
8:   else
9:     Split  $T$  into four mid-point subtriangles  $\{T_1, T_2, T_3, T_4\}$ 
10:    Add non-empty subsets  $(T_i, P \cap T_i)$  to  $\mathcal{A}$ 
11:   end if
12: end while
13: return MergeUnderfilledClusters( $\mathcal{C}, k$ )

```

into four subsets, resulting in a partitioning tree depth of $\log_4(N)$. The implementation uses vectorised NumPy operations to evaluate barycentric coordinates. The algorithm therefore, performs the processing of all points at any given depth in $O(N)$ time. As a result, for the average case, time complexity is bounded at $O(N \log N)$. The worst case space complexity is $O(N)$.

3.4.2 Preventing Overclustering

While triangular partitioning aligns well with the color code’s topology, its strict four-way splitting often produces sparse, “underfilled” clusters near the lattice boundaries. These isolated clusters severely increase inter-trap routing overhead, leading to prolonged execution times and higher logical error rates.

To mitigate this, Algorithm 2 implements a capacity-aware, bottom-up merging phase. The procedure iteratively sorts clusters by size and attempts to merge the smallest underfilled clusters with nearby neighbours, provided the combined ion count does not exceed the hardware’s maximum trap capacity. The algorithm terminates when no further merges are possible.

To optimise the selection of merge candidates, three distinct neighbour-search strategies were eval-

uated:

- **Bounded distance:** Evaluates neighbours strictly within a threshold radius. To preserve spatial locality, the threshold is typically set near the maximum distance between adjacent ancilla qubits, $\text{thresh} \approx \max_{i,j} \text{distance}(a_i, a_j)$.
- **Unbounded distance:** Evaluates all available clusters across the lattice (equivalent to the bounded strategy as $\text{thresh} \rightarrow \infty$).
- **k -nearest-neighbour (k-NN):** Restricts the candidate search space to the k geometrically closest clusters.

Algorithm 2 Heuristic Merging of Underfilled Clusters

```

1: Input: Set of initial clusters  $\mathcal{C}$ , maximum capacity  $k_{\max}$ 
2: Output: Set of capacity-optimised clusters  $\mathcal{C}$ 
3: repeat
4:    $\text{mergedThisRound} \leftarrow \text{False}$ 
5:   Sort clusters in  $\mathcal{C}$  in ascending order of their size  $|c|$ 
6:   for each cluster  $c_i \in \mathcal{C}$  do
7:     Identify neighbouring clusters  $\mathcal{N}(c_i)$  {e.g. via threshold radius or  $k$ -NN}
8:     Filter  $\mathcal{N}(c_i)$  for valid candidates where  $|c_i| + |c_j| \leq k_{\max}$ 
9:     if valid candidates exist then
10:      Select candidate  $c_{\text{best}}$  with the minimum centre-to-centre distance to  $c_i$ 
11:      Create merged cluster:  $c_{\text{new}} \leftarrow c_i \cup c_{\text{best}}$ 
12:      Compute updated centre of mass:  $\mu_{\text{new}} \leftarrow \frac{|c_i|\mu_{c_i} + |c_{\text{best}}|\mu_{c_{\text{best}}}}{|c_i| + |c_{\text{best}}|}$ 
13:      Replace  $c_i$  and  $c_{\text{best}}$  in  $\mathcal{C}$  with the combined  $c_{\text{new}}$ 
14:       $\text{mergedThisRound} \leftarrow \text{True}$ 
15:    break
16:   end if
17: end for
18: until not  $\text{mergedThisRound}$ 
19: return  $\mathcal{C}$ 

```

A key data structure in this implementation was the use of a KD-tree, a binary search tree in K -dimensional space with N items, enabling fast $O(\log N)$ lookups of cluster centres for operations relating to line 6 in Alg. 2.

Suppose the input to the merging algorithm is K clusters. Analysing a single iteration of the while loop, initialisation of a KD-tree takes $O(K \log K)$. The sorting of the active clusters by size also takes $O(K \log K)$. The KD-tree is then queried for the nearest neighbour(s) (depending on the strategy), resulting in a query time of at most $O(K \log K)$. In the worst case scenario, the algorithm finds one valid merge per while-loop iteration, requiring a rebuild of the KD-tree each iteration, causing the loop to run $O(K)$ times and producing a total time complexity of $O(K^2 \log K)$. Considering that K is often significantly smaller than N , where N is the number of qubits, the effect of the merging process on the execution time of the overall clustering mechanism is almost negligible.

A comprehensive breakdown of the results and the effects of the varying clustering strategies is discussed in Section 4.3.

3.4.3 Edge Case Mapping Methodology

One observation in the development stages of Section 3.4 was that the underlying cluster-trap assignment would result in a configuration that reduced potential parallelisation for the (6.6.6) color code at a trap capacity of 2. An alternative mapping for this edge case is proposed, which preserves the topology of the code and directly maps it to corresponding traps in a grid architecture.

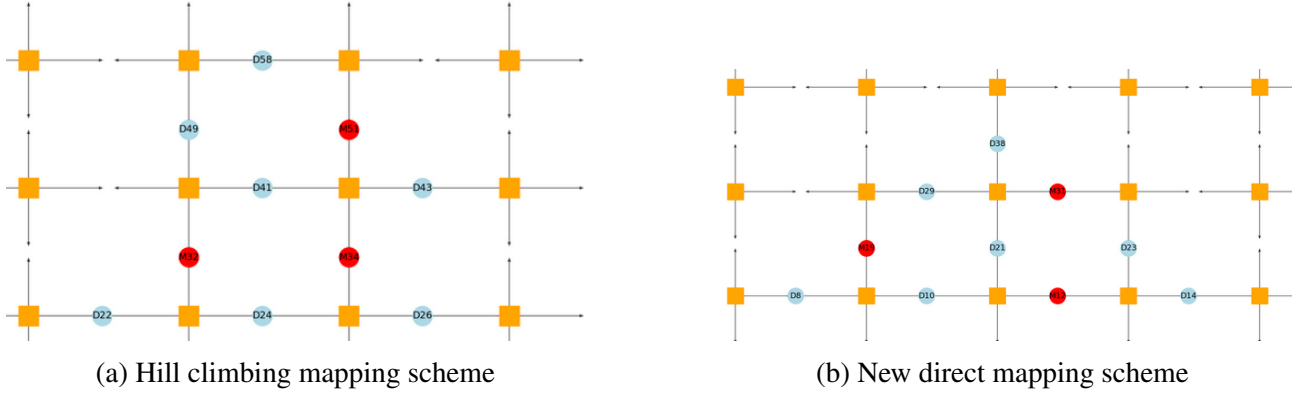


Figure 3.8: Comparison of the baseline mapping scheme alongside the proposed mapping scheme, shown on a distance 3 (6.6.6) color code.

3.5 Testing and Debugging

During the initial color code construction (Section 3.3), an SVG generation tool was implemented to visually map the code topology and verify qubit coordinate placements. The resulting topology was then validated against the expected topology and diagrams in Refs. [21, 23]. The structural integrity of the resulting quantum circuits was validated using JupyterNotebook sessions alongside Crumble, an interactive circuit visualisation tool within the `stim` ecosystem [39].

The hardware mapping and clustering logic (Section 3.4) were debugged using detailed execution logs and cluster visualisations in the simulator. To guarantee the robustness of the mapping strategies, a suite of unit tests was developed using the `pytest` framework, ensuring the methods functioned correctly under expected operations and unexpected edge cases.

The overall compilation pipeline was built with a logger, outputting errors alongside progress indications into a logfile produced on each batched experiment run.

3.6 Extension: The (4.8.8) Color Code

Alongside the (6.6.6) color code, a stretch goal included the implementation of an alternative color code tessellation to provide a richer source of testing material to further validate the compiler and mapping methodologies.

The implementation of this extension was exceedingly difficult relative to the (6.6.6) code, spanning a development of over 4 months alongside initial plans to abandon the extension. The main factor of this difficulty was the scarcity of documentation on the (4.8.8) code available as a reference when implementing the circuit. For example, the pre-existing `stim` library `ColorCodeStim` supports

neither the (4.8.8) tessellation nor chromobius compatible stim circuits [54] and papers often only detailed the stabilizer formalism of the code [29].

The initial challenge in implementing this color code was generating a tiling for its geometry, which deviates from the uniformly tiled hexagons of the (6.6.6) color code. In addition to the difficulty of generating a single tiling, another requirement was that the tiling mechanism had to generalise to all possible input distances.

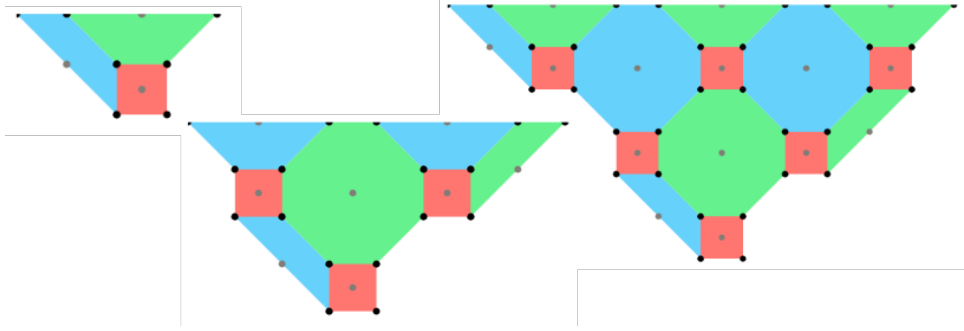


Figure 3.9: Generated tilings of the (4.8.8) triangular color code at distances 3, 5 and 7 (from left to right).

The second challenge lay in the generation of the quantum circuit, where the syndrome extraction stage required finer detail, relative to the (6.6.6), due to different weight-4 and weight-8 stabilizers affecting CNOT gate schedules. The exact CNOT schedule for the parity measurements was developed manually and built into the circuit building stage. Similarly to the (6.6.6) implementation, the circuit construction was also implemented with circuit-level noise (detailed in Section 3.3.3).

Further difficulties arose with the definition of the logical observable. Building upon the foundational properties of logical observables established in Section 3.4 (i.e., the strict requirement that $\bar{L} \in N(\mathcal{S}) \setminus \mathcal{S}$), identifying the observable for the (4.8.8) color code, due to its peculiar topology, involved a more rigorous approach than hardcoding the qubits of an edge as done in the (6.6.6) implementation. To programmatically define a valid boundary observable, the implementation defines the `_get_topological_logical_observable(colour)` routine. This function isolates every data qubit that does *not* belong to a specific colour’s face, e.g. with `colour = red`, systematically collecting the qubits positioned along the non-red (green and blue) boundaries. The resulting Pauli operator, $\bar{L}_Z = \bigotimes_{v \notin R} Z_v$ acts as a valid logical observable for the following reasons:

- **Commutativity:** The operator’s support is entirely disjoint from all red X -stabilizers, which guarantees trivial commutation. For green and blue X -stabilizers, any overlapping boundary qubits appear in pairs, ensuring an even intersection that preserves commutativity.
- **Non-Triviality:** The selected qubits form an unbroken chain that spans continuously from the green boundary to the blue boundary without touching the red boundary. As a result, it cannot be decomposed into some product of localised stabilizers and is therefore non-trivial.

Defining the logical observable this way enabled flexibility in the boundary choice when experimenting with logical error rate calculations for the circuit.

3.7 Extension: Research-based Experiments

On completion of the compilation pipeline for the color code, the framework is now able to process and evaluate the suitability of the code on various architectural QCCD designs. Naturally, this opens up the project towards researching some key questions.

- *How does the choice of trap capacity influence logical error rates?*
- *What parameters or properties are the compilation time bottlenecks for color codes?*
- *What specific architectural parameters provide the most promising path towards FTQC with color codes?*
- *How do color code performances compare with surface codes on the same architectures?*

For this section, data collection scripts were written to generate and accumulate the required experimental data. Results were subsequently ported into JupyterNotebook for its interactive features in analysing the outcomes and generating figures.

Rather than interleaving experimental results within this chapter, **the subsequent Evaluation chapter includes a comprehensive breakdown of the insights gathered in this deliverable** (from Section 4.6). With this in mind, the setup and methodology of the extension work are also included within the next chapter.

3.8 Extension: Generalising Ion Routing

Ion routing is required in the pipeline to translate an input `stim` circuit instructions into a routing schedule, necessary for ions to be co-located in the same trap to perform two-qubit operations. The output is given as a list of QCCD qubit and routing operations, along with barriers to specify operation ordering. The original algorithm was built on a key assumption that all two-qubit operations involve one ancilla and one data qubit (AD interactions) [17]. Whilst this holds for the surface code and the single-ancilla color codes, this prevents circuits such as the superdense color code, involving dual ancilla operations, from being compiled [22].

An alternative routing generalising for ancilla-ancilla/data-data (AA/DD) interactions was developed whilst avoiding drastic changes to the AD routing logic. The following key changes were made:

- Removing the hardcoded AD assumption when selecting the routing qubit.
- New generalised routing ion selection heuristic
- Guarding the ions involved in AA/DD interactions such that the routing ion is not expected to be stationary for another schedule and vice versa
- Progress detection for deadlock mitigation

The algorithm is given in Appendix A.1 together with a compilation sweep over a wider variety of QEC codes that benefit from this routing scheme in order to verify the intended functionality. The alternative routing algorithm was integrated into the compilation pipeline, offering a choice between the two algorithms in building a circuit schedule that is subsequently parallelised and simulated via `stim`.

Chapter 4

Evaluation

This chapter evaluates the performance, validity, and architectural implications of the QCCD compilation framework developed in this project. The evaluation is structured into two primary phases. The first phase validates the core project objectives, validating the color code constructions, analysing the efficacy of the proposed mapping strategy and benchmarking the overall compiler performance. The second phase explores the outcomes of the implementation work from Section 3.7.

4.1 Evaluation Methodology

4.1.1 Architectural Setup

The communication topologies explored were the grid topology, consisting of traps in a squared 2-D mesh, the linear topology, composed of traps in a 1-D chain and a networked switch topology, consisting of traps connected to a central routing segment. Linear traps resemble devices from Quantinuum’s H-Series [46], whilst the switch topology is modelled after MUSIQC, a proposed modular TI architecture [55]. The experiments range over trap capacities from 2 to 120 to explore QCCD designs ranging from highly modular (smaller traps) QCCD architectures anticipated in the near future, established by Kielpinski et al. [43] and demonstrated in commercial architectures [46, 56], alongside monolithic, high-capacity trap designs [44, 45].

The architecture simulations include a gate-improvement factor, included to model the fidelity improvements of future devices. For example, a 5x gate improvement corresponds to 5x lower error rates for gates and idle qubits, roughly in the order of 10^{-3} , depicting rates akin to existing QCCD devices [3].

Table 4.1 details the noise model used for the architecture, closely following the initial model from Ref. [29] and later adapted into the underlying architecture framework with minor deviations Ref. [17]. The compilation produces a circuit with the architectural noise model that is subsequently simulated in `stim` with the `chromobius` decoder. The operation timings are utilised in later benchmarking stages to calculate the minimum theoretical elapsed timings of circuit-hardware pairings.

Color code circuits were compiled onto hardware ranging over code distances, communication topology, trap sizes and gate improvements. Simulations were subsequently run with `stim` to assess the logical error rate.

QCCD Operation	Duration	Infidelity
(t1) Two-qubit MS gate	$40\mu s$	Combined noise params from [17]
(t2-t4) Ion Rotation	$5\mu s$	Combined noise params from [17]
(t5) Measurement	$400\mu s$	1×10^{-3}
(t6) Qubit reset	$50\mu s$	5×10^{-3}
(t7) Ion shuttling	$5\mu s$	$\bar{n} < 0.1$
(t8-t9) Ion split and merge	$80\mu s$	$\bar{n} < 6$
(t10-t11) Junction entry/exit	$100\mu s$	$\bar{n} < 3$

Table 4.1: The pessimistic upper-bound architectural noise model used in the circuit simulation after compilation, from [17]. With idling noise for operations (t7-t11) modelled with the mean vibrational energy $\bar{n}$.

4.1.2 Clustering Heuristics

As established in Section 3.4, the physical viability of color codes on QCCD architectures hinges on the efficiency of mapping their topology to hardware traps. The formed clusters of qubits dictate the required ion routing schedules, thereby impacting the overall compilation overhead and final logical error rates. The experiments evaluate the operational impact of the three algorithmic variants of the triangular clustering: bounded distance merging, unbounded distance merging and k-nearest-neighbour merging. The baseline used for comparison is the iterative bisection clustering used for surface codes in Ref. [17]. Simulations for each clustering heuristic are run across varying color code distances and QCCD hardware configurations.

4.1.3 QEC codes

In the compilation experiments, the codes used are the (6.6.6) color code and the (4.8.8) color code. Code distances between 3 and 11 are explored, with the trade-off of higher code distances (d) exponentially reducing logical errors at the expense of $O(d^2)$ qubits. The experiments consider memory experiments, which involve one or more rounds of syndrome extraction. This is used as a benchmark to test the code's ability to maintain the logical state under architectural noise (Table 4.1). For benchmarking the logical error rates of the un-compiled color code circuits, `sinter` was used to collect repeated `stim` experiments per distance, per physical error rate under the inbuilt uniform circuit-level noise, then fitted to threshold equations detailed in the next section.

4.1.4 Metrics

Cross Threshold $p_{th}^\times$: this metric identifies a critical phase transition in the code's behaviour, where at physical error rates above the threshold, increasing code distance will introduce more physical errors, worsening the logical error rate. Conversely, physical error rates below this threshold exponentially suppresses logical errors as code distance is increased. Visually, this is the point at which the logical error curves of different code distances cross one another.

To extract this point numerically, logical error rates are fitted to the equation to find $p_{th}^\times$ [21, 57]:

$$p_L = A + B(p - p_{th}^\times)d^{1/v_0} + C(p - p_{th}^\times)^2d^{2/v_0} \quad (4.1)$$

Scaling Threshold p_{th}^* : While the cross threshold identifies the theoretical phase transition, practical FTQC is thought to operate at physical error rates significantly below this point. To benchmark the performance of the color code circuit at the operational FTQC level, where physical error rates are sufficiently below the cross threshold, the logical error rate is expected to scale exponentially according to the relation [23, 58]:

$$p_L = c \left(\frac{p}{p_{th}^*} \right)^{\alpha \left(\frac{d+1}{2} \right)} \quad (4.2)$$

Logical Error Rate: The logical error rate is used as a measure of a code’s error suppression capabilities, with the aim of suppressing logical errors below physical errors. We aim to identify the architectural and compilation parameters that achieve lower logical error rates.

Routing Overhead: The number of additional routing operations performed on the qubits, including shuttling, merge/split into traps, junction entry/exit and gate-swaps within traps. As routing introduces time delays and accrues physical errors [12, 13], the experiments aim to identify architectures that have the potential to provide fault-tolerant scaling of a color code, especially at larger code distances, despite an increased number of parity checks and therefore movement operations.

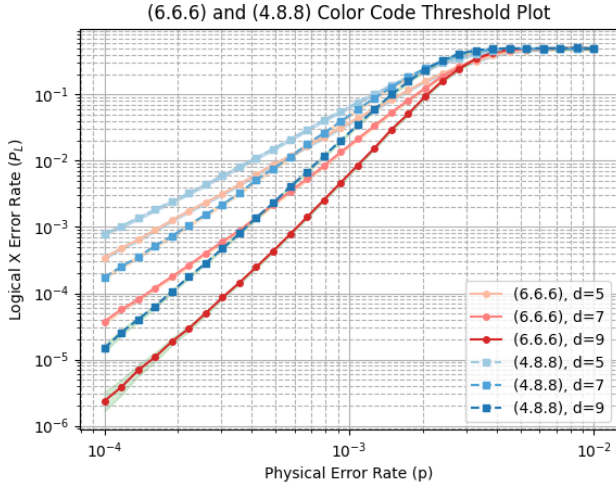
4.2 Color Code Evaluation

The logical error rates achieved from the two tessellations are shown in Fig. 4.1. Each tessellation behaves as expected, with higher distances demonstrating increased error suppression capabilities. Notably, the (6.6.6) code outperforms the (4.8.8) code at each distance. This performance discrepancy is attributable to the maximum stabilizer weights that occur in each geometry. The (4.8.8) tessellation incorporates octagonal faces that require weight-eight parity checks, leading to a CNOT scheduling depth of eight during syndrome extraction. The deeper syndrome measurement circuits in the (4.8.8) implementation inherently introduce more physical fault locations and extended error propagation paths, escalating the accumulation of physical noise compared to the structurally shallower (6.6.6) implementation. This difference in performance agrees with previously observed or hypothesised outcomes [21, 23].

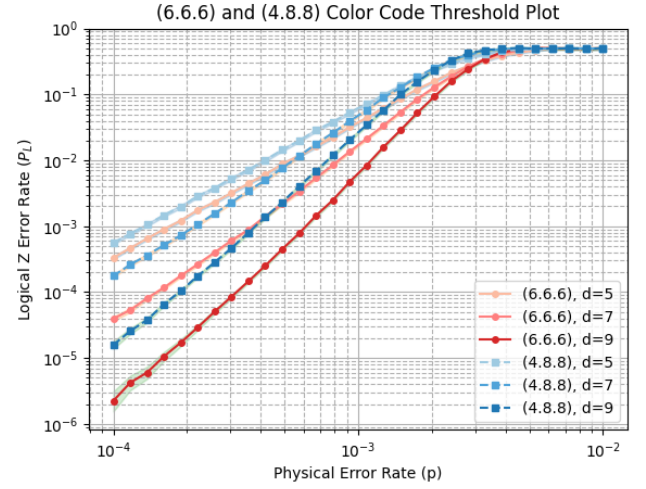
		p_{th}^*	$p_{th}^\times$
Logical X	Baseline	0.00274(6)	0.002547(8)
	Own Impl.	0.00373(1)	0.00303(4)
Logical Z	Baseline	0.00270(6)	0.002528(8)
	Own Impl.	0.00371(1)	0.00308(4)

Table 4.2: Fitted thresholds for the (6.6.6) single-ancilla color code implementation from the baseline paper [23] and this project’s implementation.

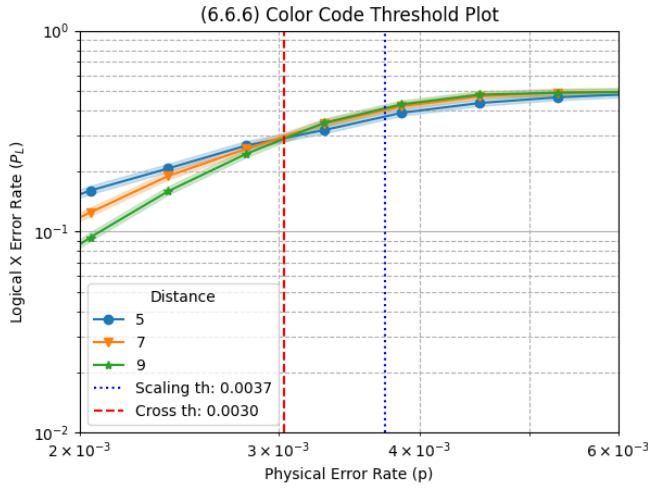
Table 4.2 holds the calculated thresholds of the (6.6.6) single-ancilla color code over the logical X and logical Z error rates beside their baseline counterpart. We achieve thresholds within the expected performance range of 0.2 – 0.47% for (6.6.6) color codes under circuit-level noise. The thresholds for the (4.8.8) color codes, given in Table 4.3, also occur between the 0.14(1) – 0.29(1)% ranges achieved in [23], with our threshold being slightly above the baseline’s single-ancilla thresholds.



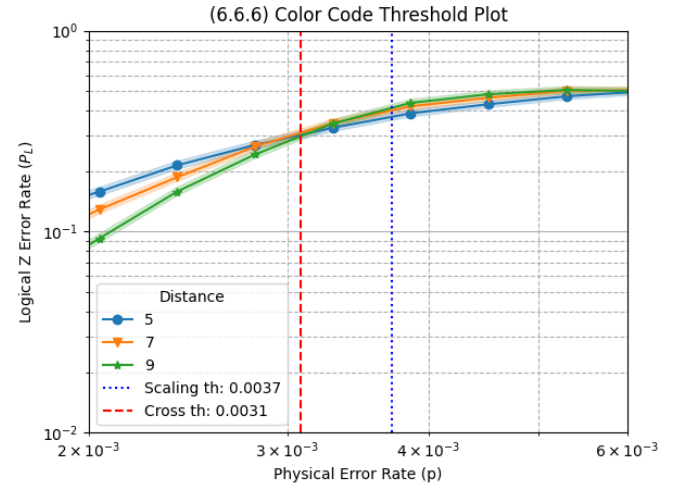
(a) Threshold plot for the logical X error rates



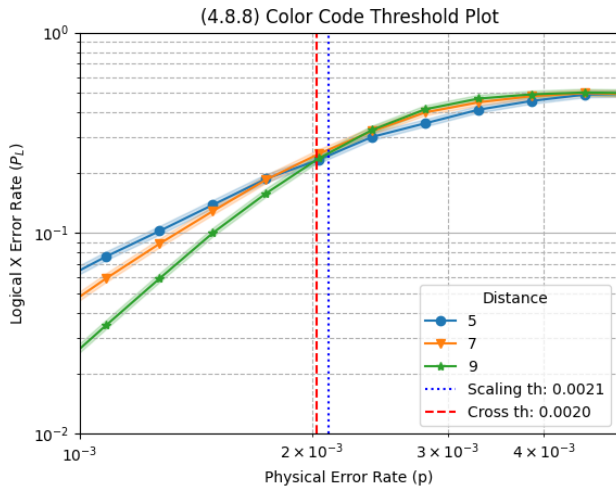
(b) Threshold plot for the logical Z error rates



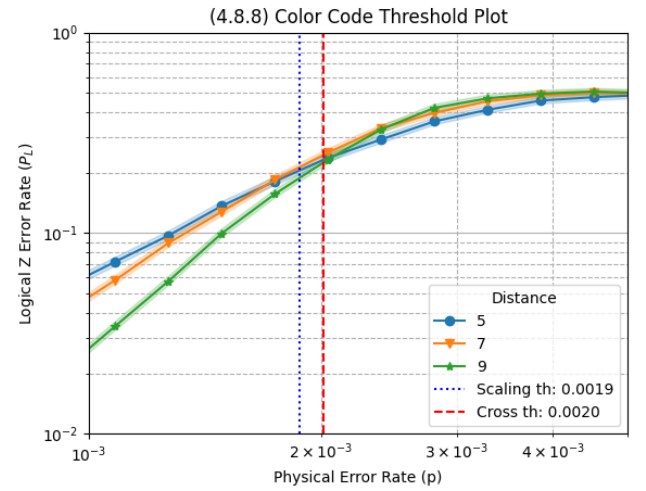
(c) Scaling and cross thresholds on (6.6.6) logical X plot



(d) Scaling and cross thresholds on (6.6.6) logical Z plot



(e) Scaling and cross thresholds on (4.8.8) logical X plot



(f) Scaling and cross thresholds on (4.8.8) logical Z plot

Figure 4.1: Threshold plots for the (6.6.6) color code implementations, with both thresholds indicated as vertical lines on the lower plots

		p_{th}^*	$p_{th}^\times$
Logical X	Baseline	0.0016(1)	0.001694(5)
	Own Impl.	0.002104(9)	0.00203(2)
Logical Z	Baseline	0.0014(1)	0.00164(2)
	Own Impl.	0.001876(8)	0.00202(2)

Table 4.3: Fitted thresholds for the (4.8.8) single-ancilla color code implementation from the baseline paper [23] and this project’s implementation.

This performance gap is likely driven by the difference in decoding methodologies. The baseline paper uses an integer programming (IP) decoder, using uniform weights for their single-ancilla circuits, implicitly using an independence assumption such that the decoder may fail to account for the impact of correlated errors, such as hook errors. The authors also explicitly note that the use of an IP decoder will produce sub-optimal decoding results. In contrast, this project uses `chromobius` which can natively resolve these correlated errors. Rather than relying on uniform-weight assumptions, the decoder utilises colour information in the circuits, allowing the detector to identify certain single-qubit faults that propagate into complex error chains [22].

In summary, the threshold plots confirm the structural integrity of the color code generation, confirming that the circuits exhibit the strict exponential error suppression that increases with distance, expected of a functional QEC scheme. Beyond base validation, both tested tessellations also achieve higher fault-tolerant thresholds than their established paper counterparts. This confirms that before any hardware-specific mapping is applied, the generated logical circuits provide a robust, fault-tolerant foundation.

4.3 Qubit Mapping

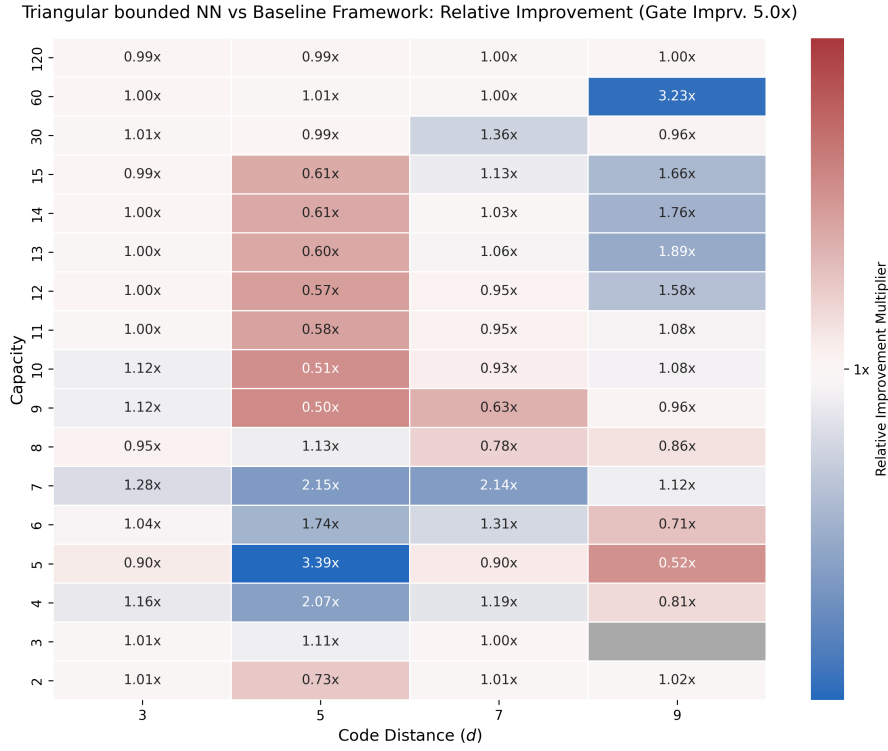
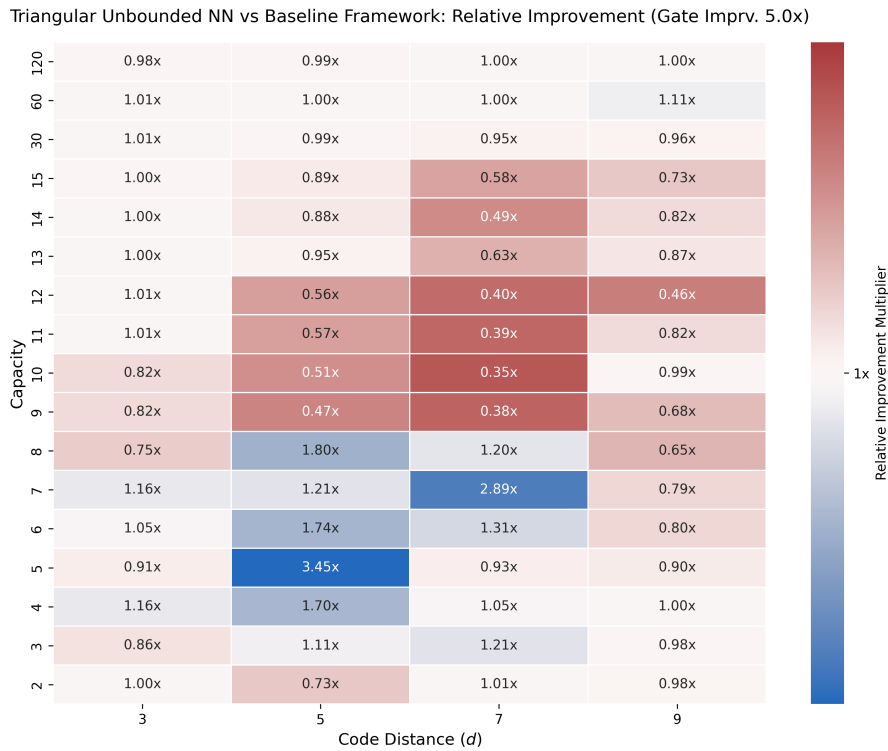
4.3.1 Clustering Effect on Compilation

As established in Section 4.2, our threshold lies in the order of 10^{-3} , so we are particularly interested in the behaviour of the schemes around and below these rates, given by gate improvements 5x and 10x.

The logical error rates achieved from experiments, using gate improvement 5x, are outlined through a comparative heatmap, coloured by the ratio between the proposed and baseline clustering schemes. Negative values (blue) indicate superior performance of the proposed algorithm for a given code distance and trap capacity.

Triangular Mapping Performance

The most critical insight derived from Fig. 4.2, Fig. 4.3 and Fig. 4.4 occurs within the resource-constrained region (trap capacities $C \leq 7$). Here, all three mapping strategies demonstrate significant error suppression capabilities, yielding up to a **3.45x improvement** over the baseline (Fig. 4.3 $d = 5$, $C = 5$). This suggests that strictly enforcing the topological locality of the color code is essential when physical hardware is highly fragmented. In the wider context of near-term QCCD development, where

Figure 4.2: Benchmark of the logical error rates of the **bounded distance scheme** against the baseline.Figure 4.3: Benchmark of the logical error rates of the **unbounded distance scheme** against the baseline.

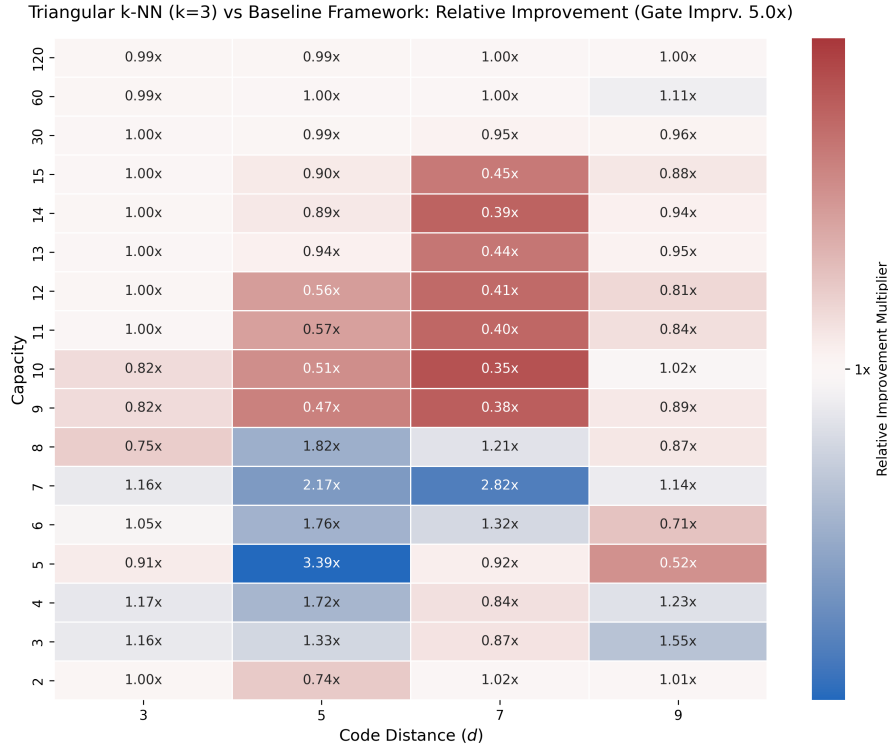


Figure 4.4: Benchmark of the logical error rates of the **k-NN scheme** against the baseline.

modular traps are strictly capacity-limited, this points to the idea that topology-aware compilation passes for topological QEC codes are highly significant in achieving lower error rates.

Interestingly, the bounded distance merging methodology also performs well in the high-capacity region for distance 9 codes, particularly around $12 \leq C \leq 15$ and $C = 60$. Whilst the baseline's greedy mechanism produces clusters maximising trap utilisation, this can ultimately reduce the available parallelism in the routing schedule. The constraints of the bounded strategy constructs smaller, geometrically regular sub-regions, which provide a schedule parallelisation advantage and retains high locality within traps. This can point to the efficacy of maintaining compact, high locality clusters when mapping larger codes onto larger hardware, rather than a greedy, capacity-maximising mapping scheme.

Clustering at Higher Capacities

While the proposed methodologies excel in constrained environments, the baseline framework retains a marginal advantage in the intermediate region where $8 \leq C \leq 15$. In this domain, the strict geometric partitioning of our triangular clustering occasionally isolates peripheral data qubits into separate traps and suffers from over-clustering, inflating the inter-trap shuttling overhead. A trade-off is seen when designing mapping methods onto hardware, where enforcing rigid topological structures offers huge payoffs in small capacity devices, but introduces routing penalties when hardware constraints are slightly relaxed.

Above a certain trap capacity (depending on the code distance), the performances of the proposed and baseline mapping schemes are roughly identical. At this point, the entire logical color code can be mapped into a single or dual-trap configuration, resulting in negligible routing overhead differences

between the algorithms.

Conclusions

The triangular clustering scheme proposed in Section 3.4 has outperformed the existing baseline heuristic at lower trap capacities and code distances, meeting the (slightly ambitious) challenge of demonstrating an efficient mapping utilising the features of a color code. Whilst the experiments ranged over code distances 3 to 9, further experiments at higher distances are required to conclusively assess the potential of triangular clustering for FTQC. The results also demonstrate that the baseline heuristic succeeds at generalising to the color code for larger trap sizes, regardless of topological differences between the surface and color codes and remains a feasible heuristic for larger trap architectures. For the most recent state-of-the-art QCCD architectures with around 100 qubits [10, 11], the architectures can support one or more color codes in the distance range 3 (10 qubits) to 9 (91 qubits), and the selection of a suitable mapping heuristic should be informed by the trap size and code distance.

4.4 Compiler Benchmarking

QEC Code	QCCD Topology	Elapsed Time (μ s) (Theoretic Min / Measured)	Number of Routing Operations (Theoretic / Measured)
(6.6.6) Distance = 3	Switch Capacity = 2	8575/8575	432/432
	Grid Capacity = 2	9490/9490	316/316
	Linear Capacity = 2	11660/18910	368/368
(4.8.8) Distance = 3	Switch Capacity = 2	9060/9060	432/432
	Grid Capacity = 2	9730/9730	344/344
	Linear Capacity = 2	14750/20675	368/368
(6.6.6) Distance = 5	Grid Capacity = 2	22495/22495	1624/1624
(4.8.8) Distance = 5	Grid Capacity = 2	32080/32080	1936/1936
(6.6.6) Distance = 9	Grid Capacity = 2	34700/34700	5916/5916
(4.8.8) Distance = 9	Grid Capacity = 2	36465/36465	4384/4384

Table 4.5: Comparison of the theoretically minimum elapsed time and routing operation overhead against the measured elapsed time, given a color code and a QCCD topology. The compiler performs optimally in the majority of cases, bar the linear topology, where several routing operations could be parallelised further.

In Table 4.5, the elapsed times for different code-hardware pairs were compared against the theoretical minimum elapsed time. **For experiments with the switch and grid topologies, the compiler achieves the theoretical minimum time.** This optimal performance is also demonstrated to scale for the grid topology for larger code distances. For the linear topology, the elapsed times were above the minimum due to a restriction within the routing algorithm, thus minimising the available parallelism. Furthermore, **the compiler is able to achieve the minimum theoretical routing overhead in all cases.**

Table 4.6 benchmarks this project’s color code compiler against QCCD compilers Moveless [14] and QCCDSim [15]. The circuits used five rounds of syndrome extraction to account for any inter-round changes. Table 4.7 quantifies the compiler’s performance via average elapsed time improvement factors across varying code tessellations and hardware topologies under the triangular bounded

clustering scheme. The data reveals a consistent performance advantage over the foundational QC-CDSim baseline, yielding average speedups ranging from 1.28x to 1.70x across all tested layouts.

D/C/T	(6.6.6) Elapsed Time (μ s)			(4.8.8) Elapsed Time (μ s)		
	This Project	QCCDSim	Moveless	This Project	QCCDSim	Moveless
3/2/L	94670	84310	75900			
5/2/L	399805	714530	408600	258915	495230	413180
3/5/L	48800	38100	34700			
5/5/L	186655	289560	103470	131550	227860	120860
3/10/L	42875	12600	12600			
5/10/L	125305	127565	51460	86155	97480	57150
3/30/L	24050	44640	44640			
5/30/L	75650	156240	156240	66450	133920	133920
3/2/G	44710	55560	41300			
5/2/G	104455	148695	181850	159600	160855	114120
3/5/G	42425	45010	41250			
5/5/G	133895	168240	111465	87615	179070	148415
3/10/G	37454	12600	12600			
5/10/G	114005	137290	63745	104915	121080	89720
3/30/G	24050	44640	44640			
5/30/G	75650	156240	156240	66450	133920	133920

Table 4.6: Benchmarking color code compilation against QCCDSim and Moveless. Each test gives the elapsed time of the circuit, specified by the code distance (D), trap capacity (C) and topology type (L = linear chains, G = grid topology) (T). The results for the (4.8.8) distance 3 circuit are omitted, as the circuit is structurally identical to the (6.6.6) distance 3 circuit, more commonly known as the Steane Code [59].

When benchmarked against the state-of-the-art Moveless framework, the proposed compiler demonstrates highly competitive circuit efficiency. For the (6.6.6) tessellation, the compiler achieves an average speedup of 1.16x on grid topologies, though it performs marginally slower (0.96x average) on linear chains. For the (4.8.8) code, the compiler consistently outperforms Moveless on average, delivering speedup factors of 1.30x on linear layouts and 1.32x on grids.

Whilst the Moveless compiler achieves the best timings for 14 out of 24 cases, compared to the 10 cases for this project’s compiler (Table 4.6), the compiler outperforms QCCDSim over 20 cases ranging in code distances, topologies and tessellations.

Under certain hardware configurations, this project’s compiler delivers maximum speedups exceeding 2.0x against both alternative frameworks. Notably, it achieves a maximum 2.07x improvement over Moveless for the (6.6.6) 5/30/L configuration, and a 2.04x improvement over QCCDSim for the (4.8.8) 5/5/G layout. These benchmarks confirm that the implemented topology-aware com-

Topology	(6.6.6) Speedup		(4.8.8) Speedup	
	QCCDSim	Moveless	QCCDSim	Moveless
Linear (L)	1.28x	0.96x	1.70x	1.30x
Grid (G)	1.31x	1.16x	1.56x	1.32x

Table 4.7: Average compilation speedup factors (Other Compiler Time / This Project Time).

piller can operate at a highly competitive standard, demonstrating a robust improvement over the foundational QCCDSim compiler.

4.5 Analysis of Direct-Mapping

Distance	Elapsed Time (μ s) Hill Climb/Direct	Number of Routing Ops. Hill Climb/Direct
3	9490/8890	316/316
5	22495/13345	1624/1176
7	26235/13345	3255/2524
9	34700/13345	5916/4416
11	36435/13345	9004/6796

Table 4.8: Comparison of the original framework’s hill climb strategy against the direct mapping proposed in Section 3.4.3.

Table 4.8 demonstrates the efficacy of the proposed direct mapping structure for the case of (6.6.6) color codes on a trap capacity 2 grid architecture, giving up to 24.5% reduction in routing operations at distance 11. Moreover, this alternative mapping scheme enables the color code to achieve constant QEC round times of 13345μ s at all code distances. The direct mapping allows preservation of the color code’s natural topology onto the QCCD traps, helping to maximise the operational parallelism.

4.6 Architectural Space Exploration

The extension work in Section 3.7 involved compilation stages over varying parameters using the triangular clustering algorithm. This section explores and answers some key questions that the extension work targets, contextualising their relevance in the trajectory of quantum error correction research. Additionally, based on the successful results of the alternative mapping in Section 4.5, the new strategy is benchmarked alongside the other hardware configurations that use the hill climbing strategy from [17].

4.6.1 Effect of Topology

As illustrated in Fig. 4.5(a), the elapsed QEC round time is highly sensitive to the underlying hardware architecture. The 1-D linear layout exhibits severe operational latency; at distance 7 capacity 2 the

round time for the linear chain reaches $\approx 173\text{ms}$, which is 6.6x and 15.5x of the grid and switch round times respectively. This degradation is largely driven by the congestion, where Fig. 4.6(b) reveals that at distance 7 and capacity 2, approximately 86% of the total cycle time is spent on inter-trap routing. Consequently, parallelisation is limited, leading to significant idling decoherence and a failure to suppress logical errors (Fig. 4.5(b)).

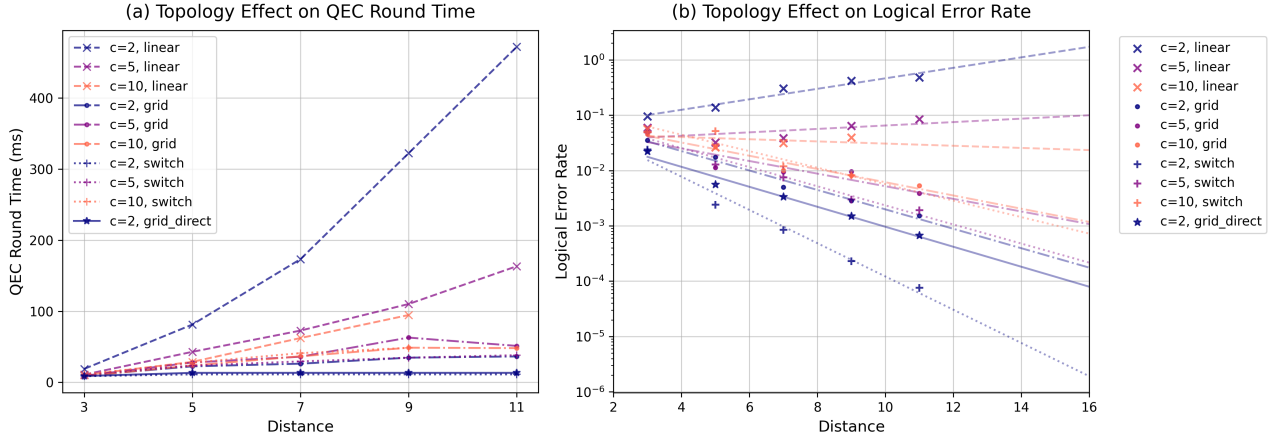


Figure 4.5: **(a)** The elapsed time over one syndrome extraction round against code distance. **(b)** The logical error rate against code distance. Both evaluated over trap capacities 2,5 and 10 using the linear, grid and switch devices at gate improvement 5x.

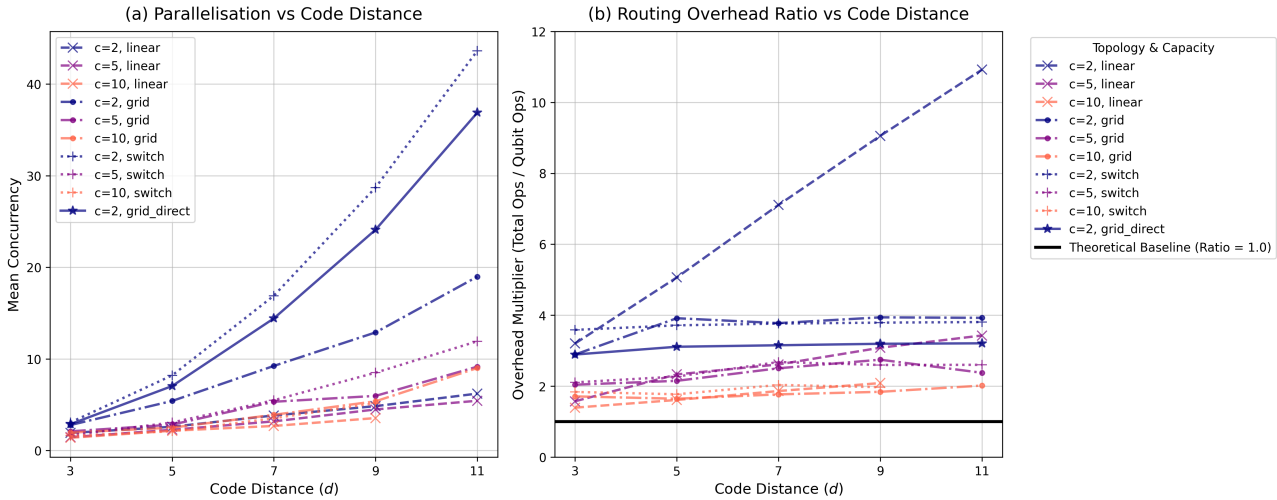


Figure 4.6: **(a)** The average number of simultaneously executed operations per QEC cycle against code distance. **(b)** The ratio between the total number of operations and the non-routing operations required to complete a QEC cycle.

Whilst moving to a 2-D grid architecture mitigates the extreme congestion of the linear chain, the default mapping strategy (hill climbing) still hinders the color code's full potential. The mapping strategy on the (6.6.6) color code can produce an unnatural mapping that minimises the potential parallelism, forcing the circuit to perform many sequential shuttling operations, accumulating further physical errors. This mismatch between the code and mapping results in constrained concurrency (Fig. 4.6(a)) and limits the grid's ability to drive down the logical error rate compared to the alternative mapping or the switch architecture.

The importance of software-hardware co-design becomes evident when comparing the default mapping to the topology-aware direct mapping scheme. As seen in Fig.4.6(b), direct mapping achieves the lowest and most consistent routing overhead of all capacity 2 configurations, maintaining a roughly constant multiplier of $\approx 3.2x$. This optimised routing produces higher concurrency (Fig.4.6(a)) and stabilises the QEC round time to a constant $\approx 13\text{ms}$. The direct mapping achieves a logical error rate of roughly 10^{-3} , outperforming the default grid mapping.

The switch network results in exceptional operational parallelism (Fig. 4.6(a)), reaching a mean concurrency of around 44 at $d = 11$. The switch topology bounds the QEC round times to a constant $\approx 11\text{ms}$. Consequently, the capacity 2 switch network achieves the lowest overall logical error rate, suppressing errors to 10^{-4} at distance 11 (Fig. 4.5(b)), making it the most robust candidate for future FTQC implementation.

A surprising insight is that in Ref. [17], compiling surface codes onto a 2-D grid architecture using a standard hill-climbing heuristic achieves fault-tolerant performance on par with the switch topology. However, this evaluation reveals that for color codes, the grid architecture fundamentally underperforms the switch—even when utilising a topology-aware mapping scheme designed for the (6.6.6) lattice. This difference highlights an evident hardware mismatch. While the regular, weight-four parity checks of a surface code naturally translate onto orthogonal grids, the trivalent structure of the color code performs better on a highly connected switch structure.

Overall, this analysis reveals a hardware-software co-design requirement, and that relying on compilation heuristics designed for surface codes can severely restrict the color code's performance. The results can be used to predict or bound the performance of future architecture designs, for example, where Fig. 4.5(b) suggests that the logical error rates of color codes on future devices potentially lie between the best performance of existing topologies (direct mapped grid) and the theoretical best (switch). Realising the full fault-tolerant potential of the color code would require a highly connected routing network, and a topology-aware compiler that can take advantage of the features of the color code.

4.6.2 Effect of Trap Capacity

After identifying the switch architecture as the most suitable candidate for FTQC on color codes, Fig. 4.7(a) reveals that a trap capacity of 2 maintains a constant QEC round time, whilst larger trap capacities exhibit cycle times scaling with code distances. Although using a smaller trap capacity incurs a higher number of routing operations, the parallelism achievable effectively offsets this overhead. Having the cycle time independent of code distance also provides the benefit of scaling to larger codes alongside predictable timing, providing a promising design paradigm for future QCCD systems.

Subfigures 4.7(b,c,d) extrapolate the logical error rates under simulated gate improvement factors of 1x, 10x and 15x. These factors correspond to approximate physical error rates of 0.009, 0.0009 and 0.0006, respectively. At 1x improvement, only the $c = 2$ configuration is able to suppress logical errors. Again, a trap capacity of 2 is the most promising architectural choice based on its error suppression ability over larger trap sizes. **Under the 15x improvement model, predictions place the required color code distance at 15 to 17 (at $c = 2$) to achieve practical logical error rates.**

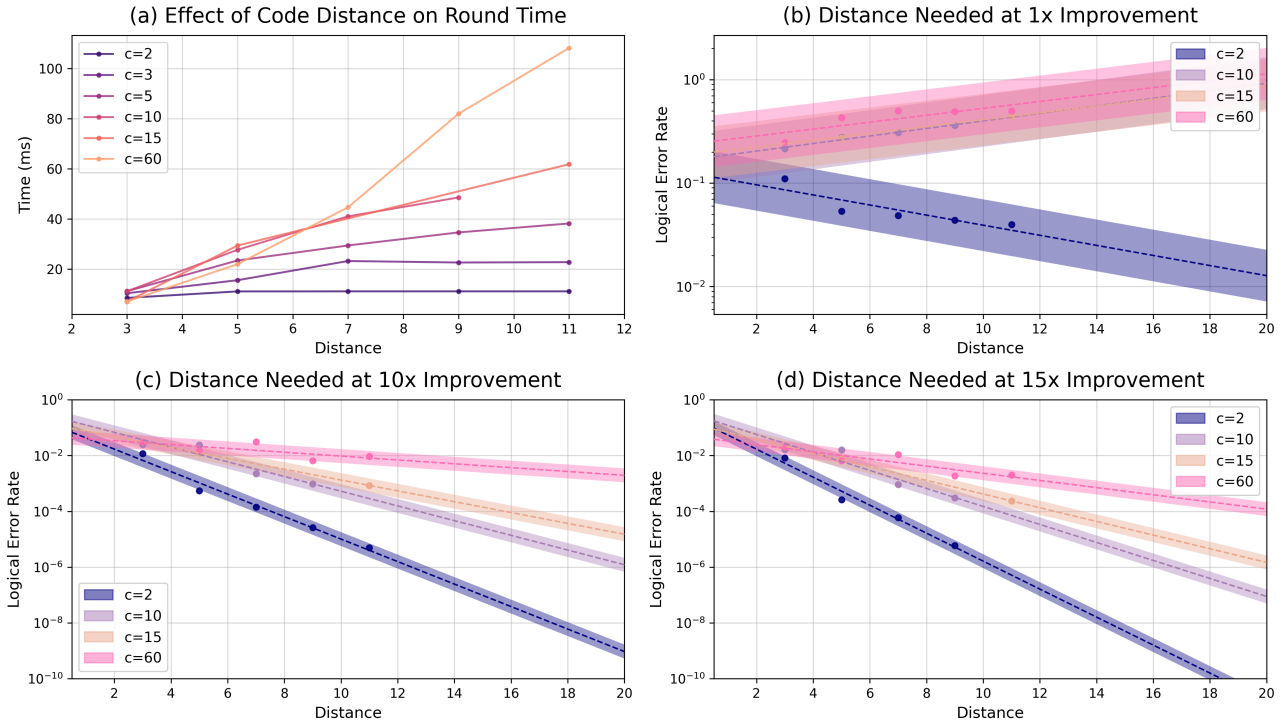


Figure 4.7: **(a)** Elapsed QEC Round Time (y-axis) against code distance (x-axis) per trap capacity. **(b,c,d)** Projections of the logical error rate of the (6.6.6) color code over varying code distances on the switch topology at gate improvements 1x, 10x and 15x. The target logical error rate for practical applications is at 10^{-10} [5], with the x-axis intercept predicting the code distance required to achieve this.

Placing these projections into the context of contemporary hardware, recent benchmarks for systems such as Quantinuum’s Helios device report physical infidelities ranging from 1.7×10^{-3} for state preparation and measurement leakage down to 3.0×10^{-5} for single-qubit gates in Quantinuum’s Helios [3]. These figures indicate that optimistically, leading architectures are already operating near the 15x improvement regime evaluated in this study. However, current physical devices remain strictly constrained by total qubit counts (98 for Helios [10]), hindering the near-term implementation of distance 15 color codes (requiring 253 qubits). To fully realise these implementations on available hardware for FTQC, further architectural improvements are required to either lower the required code distance to achieve practical logical error rates, or to increase the number of qubits available for higher code distances.

4.7 Surface vs Color Codes

As a final benchmark, the top two architectural configurations for the color code (switch and the direct-mapped grid at trap capacity 2) were compared against the top architectural configuration for surface codes, identified in Ref. [17] as the capacity 2 grid.

Figure 4.8 reveals the current performance gap between surface and color codes on QCCD hardware. The surface code maintains a constant QEC round time of approximately 4 ms, operating 3.25x and 2.75x faster than the color code on grid and switch topologies, respectively. The main factor contributing to this success being the surface code’s lower-weight parity checks and reduced circuit

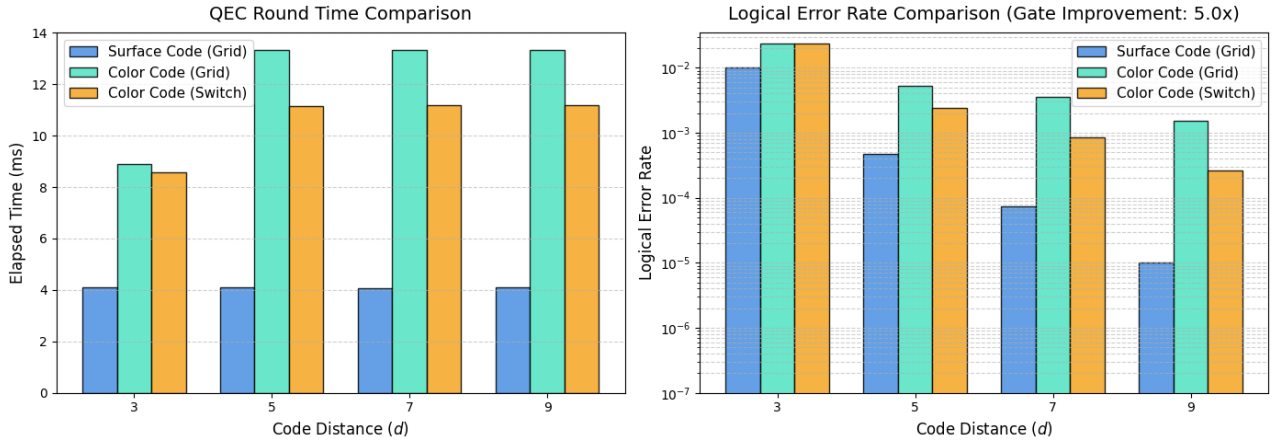


Figure 4.8: Bar charts depicting the performance of surface code on grid and the (6.6.6) color code on grid and switch architectures at trap capacity 2 for (a) Elapsed QEC Round time and (b) Logical Error Rates.

depth. Consequently, the surface code exhibits superior error suppression, achieving a logical error rate of 10^{-5} at distance 9, whereas the color code on the optimal switch network achieves a rate just above 10^{-4} .

It should be noted that this baseline comparison evaluates a single-ancilla (6.6.6) implementation. Advanced fault-tolerant variants, such as flagged-weight color codes, have demonstrated significant improvements over single-ancilla schemes [23]. Future architectural research must investigate whether these advanced variants can effectively close the observed performance gap.

Ultimately, for color codes to serve as a viable foundation for FTQC on trapped-ion hardware, either hardware and algorithmic optimisations must reduce this gap, or the system-level benefits of the color code – such as transversal Clifford gates or lower physical qubit count – must demonstrably outweigh the associated routing and error overheads. By delivering a generalised, topology-aware compilation framework, this project provides the essential infrastructure required to benchmark these advanced codes and empirically explore these exact architectural questions.

Chapter 5

Conclusion

5.1 Review of Deliverables and Contributions

Over the course of the project, all main deliverables (Table 2.1) were completed. Moreover, all intended extensions were implemented successfully. The main contributions of the project are as follows:

- The development of chromobius-compatible color code stim circuits, previously unsupported by the stim circuit library and similar works [39, 50].
- The completion of a color code compilation pipeline, rivalling the state-of-the-art Moveless compiler and provides optimal performance for the majority of code-hardware pairings. The compiler can be used to critically analyse and compare the suitability of several architecture and QEC code pairings, informing future design choices for FTQC on QCCD hardware.
- The proposal of a novel, topology-aware mapping scheme for the color code, demonstrating significant improvements over the generic baseline heuristic at low trap capacities. This strategy was previously unseen in color-code-to-QCCD capable compilers [14], and its success presents future research opportunities for QEC-aware compiler optimisations.
- Identifying the architectural parameters required to achieve practical logical error rates on future devices with color codes.
- Overall generalisation of the compilation process by decoupling the surface-code-centric assumptions inherent in the original framework, enabling a wider range of QEC codes to be evaluated on QCCD hardware.

The project will be open sourced on GitHub to enable access to the color code circuits alongside the compiler as the repository has potential impact outside of the tripos for further QEC-to-QCCD-related research.

5.2 Reflections

The difficulty of the project arose from the choice of a niche and under-researched domain. Scarcity of prior resources and literature on color code compilation introduced a steep learning curve during the preparation stage and difficulties in the implementation phase, particularly when handling undocumented edge cases or complex error mechanisms in the code or compilation pipeline. However, this

also allowed the extensions of the project to become highly research-based.

Despite the successful implementation of all deliverables, the risk associated with this project — particularly that of not being able to implement a successful mapping solution or complete the compilation pipeline — was incredibly high in hindsight, especially for the scope of a Part II project. As a result, certain implementation milestones were delayed by the unexpected complexity of the tasks and were managed by allowing large buffer spaces in the development timeline.

The choice to work with an existing codebase provided a chance to develop crucial skills such as navigating and understanding another programmer’s code and writing modular programs that can seamlessly integrate into the pipeline. This closely reflects the day-to-day experience of a software engineer, where code must be written to safely modify existing repositories, preparing me for similar working environments in the future.

5.3 Future Work

Future work could focus on expanding the framework’s interface to more easily investigate the ideal parameters for a variety of QEC code-hardware pairings that maximise each code’s potential. This could include adopting further decoders, QEC codes and platform types into the compilation pipeline. Additionally, the color code implementation could be extended to include more complex implementations that aid in increasing the error suppression capabilities of the code such as dual-ancilla or flagged syndrome extraction schemes [22, 23]. The number of available device topologies could be increased, with new designs such as the ring paradigm [48].

Bibliography

- ¹P. Shor, “Algorithms for quantum computation: discrete logarithms and factoring”, in Proceedings 35th annual symposium on foundations of computer science (1994), pp. 124–134.
- ²O. Leimkuhler and K. B. Whaley, *Exponential quantum speedups for near-term molecular electronic structure methods*, 2026.
- ³Quantinuum, *Performance validation — quantinuum systems documentation*, Accessed: 2026-04-06, 2025.
- ⁴M. DeCross, R. Haghshenas, M. Liu, E. Rinaldi, J. Gray, Y. Alexeev, C. H. Baldwin, J. P. Bartolotta, M. Bohn, E. Chertkov, J. Cline, J. Colina, D. DelVento, J. M. Dreiling, C. Foltz, J. P. Gaebler, T. M. Gatterman, C. N. Gilbreth, J. Giles, D. Gresh, A. Hall, A. Hankin, A. Hansen, N. Hewitt, I. Hoffman, C. Holliman, R. B. Hutson, T. Jacobs, J. Johansen, P. J. Lee, E. Lehman, D. Lucchetti, D. Lykov, I. S. Madjarov, B. Mathewson, K. Mayer, M. Mills, P. Niroula, J. M. Pino, C. Roman, M. Schechter, P. E. Siegfried, B. G. Tiemann, C. Volin, J. Walker, R. Shaydulin, M. Pistoia, S. A. Moses, D. Hayes, B. Neyenhuis, R. P. Stutz, and M. Foss-Feig, “Computational power of random quantum circuits in arbitrary geometries”, *Physical Review X* **15**, 10.1103/physrevx.15.021052 (2025).
- ⁵R. Acharya, D. A. Abanin, L. Aghababaie-Beni, I. Aleiner, T. I. Andersen, M. Ansmann, F. Arute, K. Arya, A. Asfaw, N. Astrakhantsev, J. Atalaya, R. Babbush, D. Bacon, B. Ballard, J. C. Bardin, J. Bausch, A. Bengtsson, A. Bilmes, S. Blackwell, S. Boixo, G. Bortoli, A. Bourassa, J. Bovaird, L. Brill, M. Broughton, D. A. Browne, B. Buchea, B. B. Buckley, D. A. Buell, T. Burger, B. Burkett, N. Bushnell, A. Cabrera, J. Campero, H.-S. Chang, Y. Chen, Z. Chen, B. Chiaro, D. Chik, C. Chou, J. Claes, A. Y. Cleland, J. Cogan, R. Collins, P. Conner, W. Courtney, A. L. Crook, B. Curtin, S. Das, A. Davies, L. De Lorenzo, D. M. Debroy, S. Demura, M. Devoret, A. Di Paolo, P. Donohoe, I. Drozdov, A. Dunsworth, C. Earle, T. Edlich, A. Eickbusch, A. M. Elbag, M. Elzouka, C. Erickson, L. Faoro, E. Farhi, V. S. Ferreira, L. F. Burgos, E. Forati, A. G. Fowler, B. Foxen, S. Ganjam, G. Garcia, R. Gasca, É. Genois, W. Giang, C. Gidney, D. Gilboa, R. Gosula, A. G. Dau, D. Graumann, A. Greene, J. A. Gross, S. Habegger, J. Hall, M. C. Hamilton, M. Hansen, M. P. Harrigan, S. D. Harrington, F. J. H. Heras, S. Heslin, P. Heu, O. Higgott, G. Hill, J. Hilton, G. Holland, S. Hong, H.-Y. Huang, A. Huff, W. J. Huggins, L. B. Ioffe, S. V. Isakov, J. Iveland, E. Jeffrey, Z. Jiang, C. Jones, S. Jordan, C. Joshi, P. Juhas, D. Kafri, H. Kang, A. H. Karamlou, K. Kechedzhi, J. Kelly, T. Khair, T. Khatkar, M. Khezri, S. Kim, P. V. Klimov, A. R. Klots, B. Kobrin, P. Kohli, A. N. Korotkov, F. Kostitsa, R. Kothari, B. Kozlovskii, J. M. Kreikebaum, V. D. Kurilovich, N. Lacroix, D. Landhuis, T. Lange-Dei, B. W. Langle, P. Laptev, K.-M. Lau, L. Le Guevel, J. Ledford, J. Lee, K. Lee, Y. D. Lensky, S. Leon, B. J. Lester, W. Y. Li, Y. Li, A. T. Lill, W. Liu, W. P. Livingston, A. Locharla, E. Lucero, D. Lundahl, A. Lunt, S. Madhuk, F. D. Malone, A. Maloney, S. Mandrà,

- J. Manyika, L. S. Martin, O. Martin, S. Martin, C. Maxfield, J. R. McClean, M. McEwen, S. Meeks, A. Megrant, X. Mi, K. C. Miao, A. Mieszala, R. Molavi, S. Molina, S. Montazeri, A. Morvan, R. Movassagh, W. Mruczkiewicz, O. Naaman, M. Neeley, C. Neill, A. Nersisyan, H. Neven, M. Newman, J. H. Ng, A. Nguyen, M. Nguyen, C.-H. Ni, M. Y. Niu, T. E. O'Brien, W. D. Oliver, A. Opremcak, K. Ottosson, A. Petukhov, A. Pizzuto, J. Platt, R. Potter, O. Pritchard, L. P. Pryadko, C. Quintana, G. Ramachandran, M. J. Reagor, J. Redding, D. M. Rhodes, G. Roberts, E. Rosenberg, E. Rosenfeld, P. Roushan, N. C. Rubin, N. Saei, D. Sank, K. Sankaragomathi, K. J. Satzinger, H. F. Schurkus, C. Schuster, A. W. Senior, M. J. Shearn, A. Shorter, N. Shutt, V. Shvarts, S. Singh, V. Sivak, J. Skrzynny, S. Small, V. Smelyanskiy, W. C. Smith, R. D. Somma, S. Springer, G. Sterling, D. Strain, J. Suchard, A. Szasz, A. Sztein, D. Thor, A. Torres, M. M. Torunbalci, A. Vaishnav, J. Vargas, S. Vdovichev, G. Vidal, B. Villalonga, C. V. Heidweiller, S. Waltman, S. X. Wang, B. Ware, K. Weber, T. Weidel, T. White, K. Wong, B. W. K. Woo, C. Xing, Z. J. Yao, P. Yeh, B. Ying, J. Yoo, N. Yosri, G. Young, A. Zalcman, Y. Zhang, N. Zhu, and N. Zobrist, "Quantum error correction below the surface code threshold", *Nature* **638**, 920–926 (2024).
- ⁶Y. Wang, M. Um, J. Zhang, S. An, M. Lyu, J.-N. Zhang, L.-M. Duan, D. Yum, and K. Kim, "Single-qubit quantum memory exceeding ten-minute coherence time", *Nature Photonics* **11**, 646–650 (2017).
- ⁷H. Zhou, C. Duckering, C. Zhao, D. Bluvstein, M. Cain, A. Kubica, S.-T. Wang, and M. D. Lukin, "Resource analysis of low-overhead transversal architectures for reconfigurable atom arrays", in *Proceedings of the 52nd annual international symposium on computer architecture, SIGARCH '25* (2025), pp. 1432–1448.
- ⁸R. Systems, *Cepheus-1-108q quantum processor*.
- ⁹M. Malinowski, D. Allcock, and C. Ballance, "How to wire a 1000-qubit trapped-ion quantum computer", *PRX Quantum* **4**, 10.1103/prxquantum.4.040313 (2023).
- ¹⁰A. Ransford, M. S. Allman, J. Arkinstall, J. P. C. III, S. F. Cooper, R. D. Delaney, J. M. Dreiling, B. Estey, C. Figgatt, A. Hall, A. A. Husain, A. Isanaka, C. J. Kennedy, N. Kotibhaskar, I. S. Madjarov, K. Mayer, A. R. Milne, A. J. Park, A. P. Reed, R. Ancona, M. P. Andersen, P. Andres-Martinez, W. Angenent, L. Argueta, B. Arkin, L. Ascarrunz, W. Baker, C. Barnes, J. Bartolotta, J. Berg, R. Besand, B. Bjork, M. Blain, P. Blanchard, R. Blume-Kohout, M. Bohn, A. Borgna, D. Y. Botamanenko, R. Boutelle, N. Brown, G. T. Buckingham, N. Q. Burdick, W. C. Burton, V. Carey, C. J. Carron, J. Chambers, J. Children, V. E. Colussi, S. Crepinsek, A. Cureton, J. Davies, D. Davis, M. DeCross, D. Deen, C. Delaney, D. DelVento, B. J. DeSalvo, J. Dominy, R. Duncan, V. Eccles, A. Edgington, N. Erickson, S. Erickson, C. T. Ertsgaard, B. Evans, T. Evans, M. I. Fabrikant, A. Fischer, C. Foltz, M. Foss-Feig, D. Francois, B. Freyberg, C. Gao, R. Garay, J. Garvin, D. M. Gaudiosi, C. N. Gilbreth, J. Giles, E. Glynn, J. Graves, A. Hansen, D. Hayes, L. Heidemann, B. Higashi, T. Hilbun, J. Hines, A. Hlavaty, K. Hoffman, I. M. Hoffman, C. Holliman, I. Hooper, B. Horning, J. Hostetter, D. Hothem, J. Houlton, J. Hout, R. Hutson, R. T. Jacobs, T. Jacobs, M. Johannsen, J. Johansen, L. Jones, S. Julian, R. Jung, A. Keay, T. Klein, M. Koch, R. Kondo, C. Kong, A. Kosto, A. Lawrence, D. Liefer, M. Lollie, D. Lucchetti, N. K. Lysne, C. Lytle, C. MacPherson, A. Malm, S. Mather, B. Mathewson, D. Maxwell, L. McCaffrey, H. McDougall, R. Mendoza, M. Mills, R. Morrison, L. Narmour, N. Nguyen, L. Nugent, S. Olson, D. Ouellette, J. Parks, Z. Peters, J. Pet-

ricka, J. M. Pino, F. Polito, M. Preidl, G. Price, T. Proctor, M. Pugh, N. Ratcliff, D. Raymondson, P. Rhodes, C. Roman, C. Roy, C. Ryan-Anderson, F. B. Sanchez, G. Sangiolo, T. Sawadski, A. Schaffer, P. Schow, J. Sedlacek, H. Semenenko, P. Shevchuk, S. Shore, P. Siegfried, K. Singhal, S. Sivarajah, T. Skripka, L. Sletten, B. Spaun, R. T. Sprenkle, P. Stoufer, M. Tader, S. F. Taylor, T. H. Thompson, R. Tobey, A. Tran, T. Tran, G. Vittorini, C. Volin, J. Walker, S. White, D. Wilson, Q. Wolf, C. Wringe, K. Young, J. Zheng, K. Zuraski, C. H. Baldwin, A. Chernoguzov, J. P. Gaebler, S. J. Sanders, B. Neyenhuis, R. Stutz, and J. G. Bohnet, *Helios: a 98-qubit trapped-ion quantum computer*, 2025.

¹¹IonQ, *Ionq tempo: 100-qubit quantum computer (#aq 64)*.

¹²A. Paetznick, M. P. da Silva, C. Ryan-Anderson, J. M. Bello-Rivas, J. P. C. III, A. Chernoguzov, J. M. Dreiling, C. Foltz, F. Frachon, J. P. Gaebler, T. M. Gatterman, L. Grans-Samuelsson, D. Gresh, D. Hayes, N. Hewitt, C. Holliman, C. V. Horst, J. Johansen, D. Lucchetti, Y. Matsuoka, M. Mills, S. A. Moses, B. Neyenhuis, A. Paz, J. Pino, P. Siegfried, A. Sundaram, D. Tom, S. J. Wernli, M. Zanner, R. P. Stutz, and K. M. Svore, *Demonstration of logical qubits and repeated error correction with better-than-physical error rates*, 2024.

¹³C. Ryan-Anderson, J. G. Bohnet, K. Lee, D. Gresh, A. Hankin, J. P. Gaebler, D. Francois, A. Chernoguzov, D. Lucchetti, N. C. Brown, T. M. Gatterman, S. K. Halit, K. Gilmore, J. Gerber, B. Neyenhuis, D. Hayes, and R. P. Stutz, *Realization of real-time fault-tolerant quantum error correction*, 2021.

¹⁴S. Khan, S. Vittal, K. Brown, and J. Baker, *Moveless: minimizing overhead on qccds via versatile execution and low excess shuttling*, 2025.

¹⁵P. Murali, D. M. Debroy, K. R. Brown, and M. Martonosi, *Architecting noisy intermediate-scale trapped ion quantum computers*, 2020.

¹⁶A. A. Saki, R. O. Topaloglu, and S. Ghosh, “Muzzle the shuttle: efficient compilation for multi-trap trapped-ion quantum computers”, in 2022 design, automation amp; test in europe conference amp; exhibition (date) (Mar. 2022), pp. 322–327.

¹⁷S. Jones and P. Murali, *Architecting scalable trapped ion quantum computers using surface codes*, 2025.

¹⁸T. Leblond, R. S. Bennink, J. G. Lietz, and C. M. Seck, “Tisc: a surface code compiler and resource estimator for trapped-ion processors”, in Proceedings of the sc ’23 workshops of the international conference on high performance computing, network, storage, and analysis, SC-W 2023 (Nov. 2023), pp. 1426–1435.

¹⁹H. Bombin and M. A. Martin-Delgado, “Topological quantum distillation”, *Phys. Rev. Lett.* **97**, 180501 (2006).

²⁰S. Koutsoumpas, T. Noszko, H. Sayginel, M. Webster, and J. Roffe, *Colour codes reach surface code performance using vibe decoding*, 2025.

²¹A. J. Landahl, J. T. Anderson, and P. R. Rice, *Fault-tolerant quantum computing with color codes*, 2011.

²²C. Gidney and C. Jones, *New circuits and an open source decoder for the color code*, 2023.

- ²³Y. Takada and K. Fujii, “Improving threshold for fault-tolerant color-code quantum computing by flagged weight optimization”, *PRX Quantum* **5**, 10.1103/PRXQuantum.5.030352 (2024).
- ²⁴A. Ovide, D. Cuomo, and C. G. Almudever, *Scaling and assigning resources on ion trap qccd architectures*, 2024.
- ²⁵IBM, *Foundations of quantum error correction*.
- ²⁶E. Knill, R. Laflamme, and W. H. Zurek, “Resilient quantum computation”, *Science* **279**, 342–345 (1998).
- ²⁷J. Preskill, “Reliable quantum computers”, *Proceedings of the Royal Society of London. Series A: Mathematical, Physical and Engineering Sciences* **454**, 385–410 (1998).
- ²⁸B. Eastin and E. Knill, “Restrictions on transversal encoded quantum gate sets”, *Physical Review Letters* **102**, 10.1103/physrevlett.102.110502 (2009).
- ²⁹M. Gutiérrez, M. Müller, and A. Bermúdez, “Transversality and lattice surgery: exploring realistic routes toward coupled logical qubits with trapped-ion quantum processors”, *Physical Review A* **99**, 10.1103/physreva.99.022330 (2019).
- ³⁰A. Kubica and M. E. Beverland, “Universal transversal gates with color codes: a simplified approach”, *Physical Review A* **91**, 10.1103/physreva.91.032330 (2015).
- ³¹E. T. Campbell, B. M. Terhal, and C. Vuillot, “Roads towards fault-tolerant universal quantum computation”, *Nature* **549**, 172–179 (2017).
- ³²B. M. Terhal, “Quantum error correction for quantum memories”, *Rev. Mod. Phys.* **87**, 307–346 (2015).
- ³³.
- ³⁴A. R. O’Rourke and S. Devitt, *Compare the pair: rotated vs. unrotated surface codes at equal logical error rates*, 2024.
- ³⁵A. M. Stephens, “Fault-tolerant thresholds for quantum error correction with the surface code”, *Phys. Rev. A* **89**, 022321 (2014).
- ³⁶D. S. Wang, A. G. Fowler, and L. C. L. Hollenberg, “Surface code quantum computing with error rates over 1%”, *Phys. Rev. A* **83**, 020302 (2011).
- ³⁷J. E. Moussa, *Recursive magic state distillation on the surface code*, 2026.
- ³⁸D. Horsman, A. G. Fowler, S. Devitt, and R. V. Meter, “Surface code quantum computing by lattice surgery”, *New Journal of Physics* **14**, 123011 (2012).
- ³⁹C. Gidney, “Stim: a fast stabilizer circuit simulator”, *Quantum* **5**, 497 (2021).
- ⁴⁰O. Higgott, *Pymatching: a python package for decoding quantum codes with minimum-weight perfect matching*, 2021.
- ⁴¹K. Sahay and B. J. Brown, “Decoder for the triangular color code by matching on a möbius strip”, *PRX Quantum* **3**, 010310 (2022).

- ⁴²N. M. Linke, D. Maslov, M. Roetteler, S. Debnath, C. Figgatt, K. A. Landsman, K. Wright, and C. Monroe, “Experimental comparison of two quantum computing architectures”, *Proceedings of the National Academy of Sciences* **114**, 3305–3310 (2017).
- ⁴³D. Kielpinski, C. R. Monroe, and D. J. Wineland, “Architecture for a large-scale ion-trap quantum computer”, *Nature* **417**, 709–711 (2002).
- ⁴⁴D. J. Wineland, C. Monroe, W. M. Itano, D. Leibfried, B. E. King, and D. M. Meekhof, “Experimental issues in coherent quantum-state manipulation of trapped atomic ions”, *Journal of research of the National Institute of Standards and Technology* **103**, 259 (1998).
- ⁴⁵R. J. Hughes, D. F. James, E. H. Knill, R. Laflamme, and A. G. Petschek, “Decoherence bounds on quantum computation with trapped ions”, *Physical Review Letters* **77**, 3240 (1996).
- ⁴⁶S. A. Moses, C. H. Baldwin, M. S. Allman, R. Ancona, L. Ascarrunz, C. Barnes, J. Bartolotta, B. Bjork, P. Blanchard, M. Bohn, J. G. Bohnet, N. C. Brown, N. Q. Burdick, W. C. Burton, S. L. Campbell, J. P. Campora, C. Carron, J. Chambers, J. W. Chan, Y. H. Chen, A. Chernoguzov, E. Chertkov, J. Colina, J. P. Curtis, R. Daniel, M. DeCross, D. Deen, C. Delaney, J. M. Dreiling, C. T. Ertsgaard, J. Esposito, B. Estey, M. Fabrikant, C. Figgatt, C. Foltz, M. Foss-Feig, D. Francois, J. P. Gaebler, T. M. Gatterman, C. N. Gilbreth, J. Giles, E. Glynn, A. Hall, A. M. Hankin, A. Hansen, D. Hayes, B. Higashi, I. M. Hoffman, B. Horning, J. J. Hout, R. Jacobs, J. Johansen, L. Jones, J. Karcz, T. Klein, P. Lauria, P. Lee, D. Liefer, S. T. Lu, D. Lucchetti, C. Lytle, A. Malm, M. Matheny, B. Mathewson, K. Mayer, D. B. Miller, M. Mills, B. Neyenhuis, L. Nugent, S. Olson, J. Parks, G. N. Price, Z. Price, M. Pugh, A. Ransford, A. P. Reed, C. Roman, M. Rowe, C. Ryan-Anderson, S. Sanders, J. Sedlacek, P. Shevchuk, P. Siegfried, T. Skripka, B. Spaun, R. T. Sprenkle, R. P. Stutz, M. Swallows, R. I. Tobey, A. Tran, T. Tran, E. Vogt, C. Volin, J. Walker, A. M. Zolot, and J. M. Pino, “A race-track trapped-ion quantum processor”, *Phys. Rev. X* **13**, 041052 (2023).
- ⁴⁷Quantinuum, *Our trapped ion quantum computers: system model h1*.
- ⁴⁸S. Khan, A. Anand, K. R. Brown, and J. M. Baker, *Cyclone: designing efficient and highly parallel qccd architectural codesigns for fault tolerant quantum memory*, 2025.
- ⁴⁹S. Jones, *Scottjones03/public-material-2025*.
- ⁵⁰M. A. Perlin, *qLDPC*, <https://github.com/qLDPCorg/qLDPC>, 2023.
- ⁵¹N. Lacroix, A. Bourassa, F. J. H. Heras, L. M. Zhang, J. Bausch, A. W. Senior, T. Edlich, N. Shutty, V. Sivak, A. Bengtsson, M. McEwen, O. Higgott, D. Kafri, J. Claes, A. Morvan, Z. Chen, A. Zalcman, S. Madhuk, R. Acharya, L. Aghababaie Beni, G. Aigeldinger, R. Alcaraz, T. I. Andersen, M. Ansmann, F. Arute, K. Arya, A. Asfaw, J. Atalaya, R. Babbush, B. Ballard, J. C. Bardin, A. Bilmes, S. Blackwell, J. Bovaird, D. Bowers, L. Brill, M. Broughton, D. A. Browne, B. Buchea, B. B. Buckley, T. Burger, B. Burkett, N. Bushnell, A. Cabrera, J. Campero, H.-S. Chang, B. Chiaro, L.-Y. Chih, A. Y. Cleland, J. Cogan, R. Collins, P. Conner, W. Courtney, A. L. Crook, B. Curtin, S. Das, S. Demura, L. De Lorenzo, A. Di Paolo, P. Donohoe, I. Drozdov, A. Dunsworth, A. Eickbusch, A. M. Elbag, M. Elzouka, C. Erickson, V. S. Ferreira, L. Flores Burgos, E. Forati, A. G. Fowler, B. Foxen, S. Ganjam, G. Garcia, R. Gasca, É. Genois, W. Giang, D. Gilboa, R. Gosula, A. Grajales Dau, D. Graumann, A. Greene, J. A. Gross, T. Ha, S. Habegger, M. Hansen, M. P. Harrigan, S. D. Harrington, S. Heslin, P. Heu, R. Hiltermann, J. Hilton, S. Hong, H.-Y. Huang, A. Huff, W. J.

Huggins, E. Jeffrey, Z. Jiang, X. Jin, C. Joshi, P. Juhas, A. Kabel, H. Kang, A. H. Karamlou, K. Kechedzhi, T. Khair, T. Khattar, M. Khezri, S. Kim, P. V. Klimov, B. Kobrin, A. N. Korotkov, F. Kostritsa, J. M. Kreikebaum, V. D. Kurilovich, D. Landhuis, T. Lange-Dei, B. W. Langley, P. Laptev, K.-M. Lau, J. Ledford, K. Lee, B. J. Lester, L. Le Guevel, W. Y. Li, Y. Li, A. T. Lill, W. P. Livingston, A. Locharla, E. Lucero, D. Lundahl, A. Lunt, A. Maloney, S. Mandrà, L. S. Martin, O. Martin, C. Maxfield, J. R. McClean, S. Meeks, A. Megrant, K. C. Miao, R. Molavi, S. Molina, S. Montazeri, R. Movassagh, C. Neill, M. Newman, A. Nguyen, M. Nguyen, C.-H. Ni, M. Y. Niu, L. Oas, W. D. Oliver, R. Orosco, K. Ottosson, A. Pizzuto, R. Potter, O. Pritchard, C. Quintana, G. Ramachandran, M. J. Reagor, R. Resnick, D. M. Rhodes, G. Roberts, E. Rosenberg, E. Rosenfeld, E. Rossi, P. Roushan, K. Sankaragomathi, H. F. Schurkus, M. J. Shearn, A. Shorter, V. Shvarts, S. Small, W. C. Smith, S. Springer, G. Sterling, J. Suchard, A. Szasz, A. Szein, D. Thor, E. Tomita, A. Torres, M. M. Torunbalci, A. Vaishnav, J. Vargas, S. Vdovichev, G. Vidal, C. Vollgraff Heidweiller, S. Waltman, J. Waltz, S. X. Wang, B. Ware, T. Weidel, T. White, K. Wong, B. W. K. Woo, M. Woodson, C. Xing, Z. J. Yao, P. Yeh, B. Ying, J. Yoo, N. Yosri, G. Young, Y. Zhang, N. Zhu, N. Zobrist, H. Neven, P. Kohli, A. Davies, S. Boixo, J. Kelly, C. Jones, C. Gidney, and K. J. Satzinger, “Scaling and logic in the colour code on a superconducting quantum processor”, *Nature* **645**, 614–619 (2025).

⁵²D. Gottesman, *Stabilizer codes and quantum error correction*, 1997.

⁵³K. Andreev and H. Räcke, “Balanced graph partitioning”, in *Proceedings of the sixteenth annual acm symposium on parallelism in algorithms and architectures*, SPAA ’04 (2004), pp. 120–124.

⁵⁴S.-H. Lee, A. Li, and S. D. Bartlett, “Color code decoder with improved scaling for correcting circuit-level noise”, *Quantum* **9**, 1609 (2025).

⁵⁵C. Monroe, R. Raussendorf, A. Ruthven, K. R. Brown, P. Maunz, L.-M. Duan, and J. Kim, “Large-scale modular quantum-computer architecture with atomic memory and photonic interconnects”, *Phys. Rev. A* **89**, 022317 (2014).

⁵⁶J. M. Pino, J. M. Dreiling, C. Figgatt, J. P. Gaebler, S. A. Moses, M. S. Allman, C. H. Baldwin, M. Foss-Feig, D. Hayes, K. Mayer, C. Ryan-Anderson, and B. Neyenhuis, “Demonstration of the trapped-ion quantum ccd computer architecture”, *Nature* **592**, 209–213 (2021).

⁵⁷C. Wang, J. Harrington, and J. Preskill, “Confinement-higgs transition in a disordered gauge theory and the accuracy threshold for quantum memory”, *Annals of Physics* **303**, 31–58 (2003).

⁵⁸A. G. Fowler, M. Mariantoni, J. M. Martinis, and A. N. Cleland, “Surface codes: towards practical large-scale quantum computation”, *Phys. Rev. A* **86**, 032324 (2012).

⁵⁹A. M. Steane, “Error correcting codes in quantum theory”, *Phys. Rev. Lett.* **77**, 793–797 (1996).

Appendices

Appendix A

Supplementary Content

A.1 Alternative Ion Routing

Algorithm 3 Ion Routing

```
1: Input: QCCD architecture  $\mathcal{A}$ ; operation sequence  $\mathcal{O}$ ; trap capacity  $c$ 
2: Output: scheduled physical operation sequence; barrier positions
3: For each two-qubit gate, assign the routed and stationary ion via DetermineRouted
4: while operations remain do
5:   Execute all gates whose ions are already co-located, respecting ion conflicts within the pass
6:   for each free routed ion with a pending non-local gate whose stationary ion is also free do
7:     Reserve the stationary ion; find a shortest path to its trap avoiding occupied crossings and
       full nodes; defer to the next iteration if none exists
8:   end for
9:   while reserved movements remain do
10:    Advance each routed ion one hop along its path
11:    if a routed ion reaches its destination then
12:      Execute the gate; if the destination trap was at capacity, route the ion back to its source
        trap
13:    end if
14:  end while
15:  Insert a barrier; raise an error if no progress was made
16: end while
```

Algorithm 4 DETERMINEROUTED($i_1, i_2, \mathcal{A}$)

```
1: Input: ions  $i_1, i_2$ ; QCCD architecture  $\mathcal{A}$ 
2: Output: routed ion  $r$ , stationary ion  $s$ 
3: if  $i_1$  and  $i_2$  are the same qubit type then
4:   return the ion with the shorter path to the other ion's trap; the other ion is stationary
5: else
6:   return the ancilla ion as routed, the data ion as stationary
7: end if
```

The main (condensed) logic of the generalised ion routing is given in Alg. 3, alongside the helper function Alg. 4 used to determine the routed ion in the schedule.

A.1.1 Compilation Tests

The resulting ion routing algorithm was tested on three main color code types: the single-ancilla, superdense and middle-out.

Superdense circuits involve ancilla-ancilla (AA) interactions, whilst the middle-out circuits contain no ancillas with only data-data (DD) operations [22]. Therefore, these circuits were suitable for testing the new algorithm, aimed at generalising for both AA and DD operations

Code Type	Ion Routing (AD) Elapsed Time (μ s) / No. Routing Ops	Alt. Ion Routing Elapsed Time (μ s) / No. Routing Ops
Single-Ancilla (6.6.6) d=3 (direct-mapped)	8890/316	8890/316
Middle-Out (6.6.6) d=3 (direct-mapped)	FAIL	24825/966
Middle-Out (4.8.8) d=5 (hill-climb)	FAIL	56725/3586
Superdense (6.6.6) d=3 (hill-climb)	FAIL	27085/1000

Table A.1: Comparison of the original [17] and new ion routing algorithms on a single round circuit of the single-ancilla, superdense and middle-out code types. Mapping methodology is chosen based on the lower-elapsed time of the two methods, providing a lower bound on the achievable statistics.

As seen in Table A.1, the original ion routing algorithm is severely restrictive, failing on the middle-out and superdense circuit types. On the contrary, the new routing algorithm succeeds in routing and compiling the statistics of all three color code types, pointing to its success in generalising for AA and DD interactions. Furthermore, it also succeeds in retaining the same routing statistics for the single-ancilla circuit as the original routing algorithm.

A.2 Clustering Results

The clustering strategies of Section 3.4 impose different geometric constraints during the merging phase, generating fundamentally distinct partitions. Fig. A.2 provides visualisation of each mechanism on the distance 5 (6.6.6) color code at trap capacity 5 and 10, grouping clusters by colour alongside their centroids (indicated by $\times$).

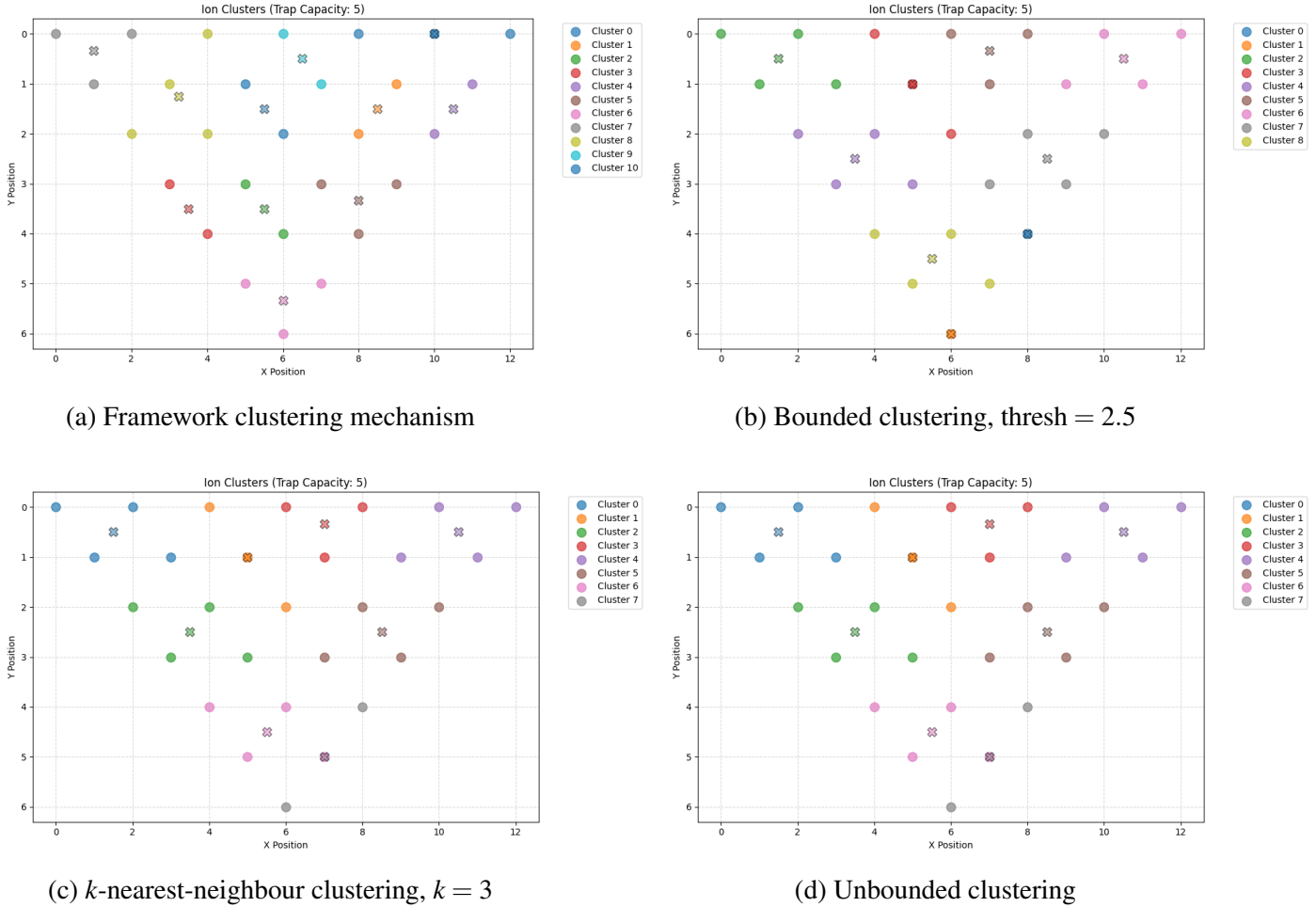
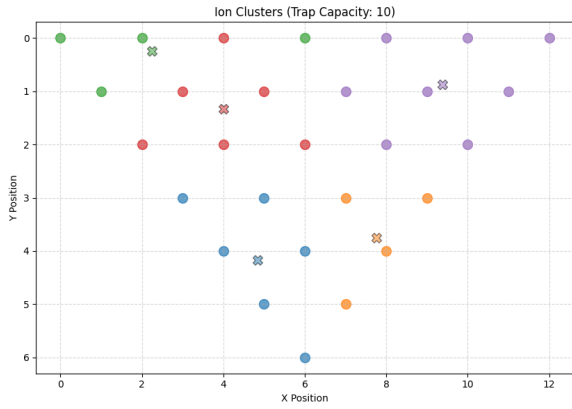


Figure A.1: Clusters formed using trap capacity of 5

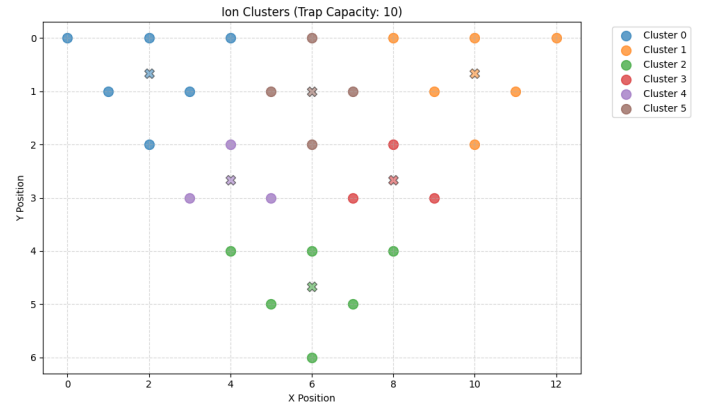
An interesting artifact of the framework clustering in Fig. A.2 (a) is observed in cluster 2, composed of three ions in the top left corner alongside a separated, singular ion towards the top middle. In comparison, the triangular clustering mechanisms preserve and form tighter clusters for the same top left corner.

This visualisation also serves as verification that the underlying triangular partitioning works as expected, obvious in the clusters formed by the bounded clustering mechanic in Fig. A.2, mirroring the intended sub-triangle formations demonstrated by Fig. 3.8(b).

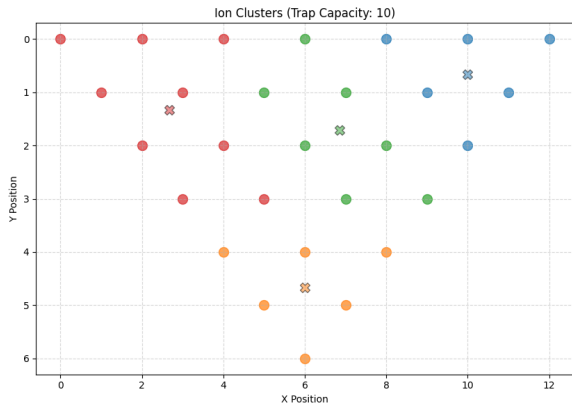
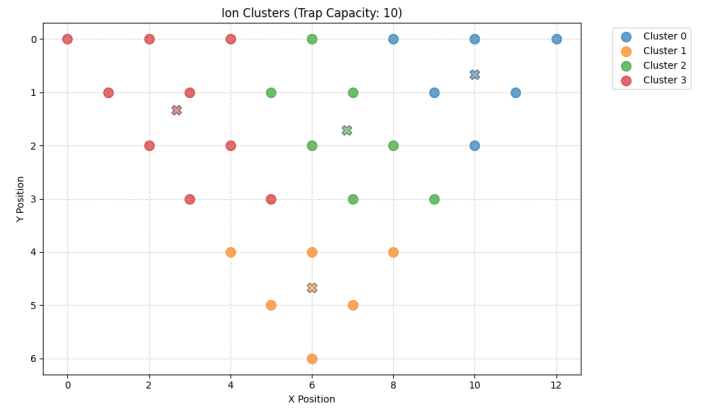
From these results, the bounded nearest-neighbour clustering demonstrates the tendency to form a larger number of clusters relative to other methods due to its tighter constraint on the search space in the merging stage. A potential avenue for future work is a parameter sensitivity test to find the optimal threshold value per color code, and whether this parameter is dependent on distance and tessellation.



(a) Framework clustering mechanism



(b) Bounded clustering, thresh = 2.5

(c) k -nearest-neighbour clustering, $k = 3$ 

(d) Unbounded clustering

Figure A.2: Clusters formed using trap capacity of 10

Appendix B

Project Proposal

B.1 Description of Project

Trapped Ion (TI) systems are a possible platform for quantum systems, of which Quantum Charge-Coupled Devices (QCCDs) are a modular architectural implementation which has demonstrated potential for Quantum Error Correction (QEC) on small TI systems [13]. Recently, a toolflow was proposed which provides a framework in which QEC codes can be compiled onto different choices of QCCD hardware to simulate and test metrics such as logical error rates [17]. While this tool provides rudimentary support for codes such as the Surface Code, compilation of Color Codes are still a niche and under-researched area. This project will focus on building a set of compiler passes for this toolbox for evaluating Color Codes as a candidate QEC code on different QCCD hardware choices.

B.1.1 Main Goals

The main goal of this project is to create a compiler that can take in QCCD hardware parameters to produce a representation of a color code that can be used to run QEC using primitive QCCD operations.

The tools/adaptations that are required are:

- A compiler pass to map logical color code qubits to QCCD ions
- A compiler pass to translate color code circuit operations to QCCD operations
- Adaptation and addition to the simulation and test infrastructure to run the compiled color code circuit

This project will be considered successful if a user can pass in a choice of QCCD hardware parameters and produce estimate logical error rates for a given color code.

Key Challenges

The main challenge of this task is to be able to leverage features of a color code to provide a suitable and ideally near-optimal mapping onto different QCCD architecture.

B.1.2 Extensions

A few stretch goals for this include some research and further tool extensions:

- Include support for different Color code tessellations e.g. [4,8,8]
- Provide mappings for similar topological codes
- Investigate the implementation and ideal QCCD hardware choices relating to higher dimensional QEC codes
- Investigate ideal QCCD hardware choices for FTQC using color codes
- Experimenting with different qubit mapping and ion routing approaches

B.2 Evaluation

Evaluation of the project will be done as follows:

- The tool for mapping the color code will be compared against how close the simulated QEC cycles gets to the optimal mapping for a pre-determined set of individual color codes and QCCD device models.
- The compiler will be evaluated with a set of QEC cycles and tested for correct termination as well as the correct sequences of gates applied to each ion from a set of color codes and QCCD device models.
- Measurement of compilation time and the number of routing operations required with comparison to theoretic times
- Evaluation of logical error rates for color code, QCCD hardware configuration pairs

B.3 Success Criteria

1. A compilation/mapping scheme for color codes onto QCCD architecture that can be used to estimate logical error rates.
2. Suitable simulation and testing infrastructure to carry out the evaluation on cycles of Color Codes on QCCD devices and the compiler.

B.4 Risk Management

As quantum computing is a complex field, one source of risk is to lack knowledge in the theory to understand or implement a part of this project. To mitigate this, I will ensure to regularly read up on literature to reinforce my knowledge and consult experts such as lecturers and my supervisor when facing any confusion.

B.5 Resource Declaration

- **Computing resources:**
 - ThinkPad T430 2-core CPU with 8GB RAM (Windows 10 and Linux) for portability

- Custom PC with a 6-core CPU and 32GB RAM (Windows 11)
- **Dissertation Backup:** Majority of my dissertation will be written and saved on Overleaf. I will also save the project onto an external hard-disk at intermediate stages.
- **Code Backup:** Code will be stored in a remote GitHub repository. Local development will be done by branching from the main code, writing code, then merged back into main.

B.6 Starting Point

Over the summer, I completed a Coursera on QEC by Google Quantum AI along with some exploration of the stim python library and the QEC test framework repository on GitHub.

B.7 Timetable

1. 10/10 - 23/10: Preparation

Deliverables:

- A comprehensive report on color codes including motivation and requirements for mapping color codes to QCCD
- Tests run on the QEC testing framework to demonstrate sufficient understanding of the current implementation

2. 24/10 - 6/11: Mapping Research

Deliverables: Research into methods of mapping qubits to ions

- Research and selection of mapping methodologies such as clustering, greedy heuristics and satisfiability (SMT)
- Report on paper readings
- Draft of preparation chapter

The algorithm should make use of the characteristics of the color code.

3. 7/11 - 4/12: Mapping Implementation

Deliverables: mappings of color codes to the available QCCD hardware choices.

- Assignment of color code qubits to physical qubits in hardware
- Adjustment and addition to framework to compile color code circuits
- Writing testing and evaluation infrastructure
- Draft of the Implementation chapter

Extended timeframe to account for 3 module assignments that take place over mid-late November.

4. 5/11 - 25/12: Extension Work

Deliverables: Implementation of any relevant extension work into core and discussion of results in relevant chapters.

5. 26/12 - 15/1: Testing + Evaluation

Deliverables: evaluating implementations and generating graphs and metrics. Draft of the evaluation chapter

Some parameters that may be tested for the QCCD QEC implementation with enough time:

- Communication topology
- Segment lengths

- Trap sizings
- Color Code tessellations
- Code dimensions
- Ancilla to data qubit ratio
- Distinct QCCD memory and processing zones

6. **16/1 - 29/1: Dissertation Intro**

Deliverable: Draft of introduction chapter. Draft of progress report due for 6/2. Send whole draft dissertation to supervisor for checks.

7. **30/1 - 19/2: Editing and Conclusion Chapter**

Deliverable: Draft of conclusion chapter. Finalising and reviewing previously drafted chapters. Progress report submission and presentation.

Slack added to account for high weekly lecture load in Lent term.

8. **20/2 - 26/3: Finalising + Editing of Chapters**

Deliverable: Full dissertation draft sent to DoS and Supervisor before the beginning of Easter break.

9. **27/3 - 15/5: Refine Final Dissertation and Submission**